МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
ФАКУЛЬТЕТ ІНФОРМАТИКИ ТА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ
КАФЕДРА ІНФОРМАТИКИ ТА ПРОГРАМНОЇ ІНЖЕНЕРІЇ

Курсова робота

з освітнього компонента «Моделювання систем. Курсова робота»

Тема: «Імітаційна модель системи посадки літаків в аеропорті на основі формального опису мережею масового обслуговування»

Дата захисту: 18 листопада 2026 р.

Керівник:
асистент кафедри ІПІ
Дмитро КОРИНЕЦЬКИЙ

Виконавець:
Андрій МЕЛЬНИЧУК
студент групи ІП-33
залікова книжка № _____

«Допущено до захисту»

_____ _____
«___» _____ 2026 р. 10 листопада 2026 р.

Захищено з оцінкою

_____

Члени комісії:
_____ Інна СТЕЦЕНКО
_____ Антон ДИФУЧИН

Київ – 2026

ЗАВДАННЯ НА КУРСОВУ РОБОТУ

Студенту групи ІП-33 Мельничуку Андрію. Керівник – асистент Коринецький Д. С.

Тема: «Імітаційна модель системи посадки літаків в аеропорті на основі формального опису мережею масового обслуговування». Варіант завдання В12, формалізм – мережа масового обслуговування, рівень складності В.

Виконати дослідження для такої системи:

Потік літаків, що потребують посадки в аеропорті, пуассонівський з інтенсивністю 10 літаків за годину. В аеропорту є дві посадкові смуги. Літак, зробивши посадку на смугу, звільняє її через 35 хвилин. Якщо літак потребує посадки, коли усі смуги зайняті, він очікує на посадку. Якщо очікування на посадку триває більше ніж 70±10 хвилин, то виникає необхідність дозаправки літака, на що аеропорту доведеться витратити 1000±200 грошових одиниць. Після 140 хвилин очікування літак відправляється на посадку в інший аеропорт. За кожний літак, що здійснив посадку без очікування, аеропорт має прибуток 2000 гривень, а за кожний, що здійснив посадку з очікуванням, – 1500±100 гривень. Витрати на будівництво додаткової посадкової смуги складають 3000000 гривень. В аеропорту не може бути побудовано більше ніж 10 додаткових смуг.

Метою моделювання є визначення кількості посадкових смуг, при якій інтервал часу їх окупності буде мінімальним, якщо час окупності додаткових посадкових смуг складає $T = S / p$, де S – витрати на побудову смуг, p – прибуток за один рік.

Для вищезазначеної системи:

1) розробити опис концептуальної моделі системи;

2) виконати формалізацію опису об'єкта моделювання в термінах визначеного у завданні формалізму;

3) розробити алгоритм імітації моделі дискретно-подійної системи у відповідності до побудованого формального опису;

4) для доведення коректності побудованого алгоритму виконати верифікацію алгоритму імітації;

5) визначити статистичні оцінки заданих характеристик моделі, що є метою моделювання;

6) провести експериментальне дослідження моделі;

7) інтерпретувати результати моделювання та сформулювати пропозиції щодо поліпшення функціонування системи;

8) зробити висновки щодо складності розробки моделі та алгоритму імітації на основі використаного формалізму, отриманих результатів моделювання та їх корисності.

Дата видачі завдання: 10 вересня 2026 р.

КАЛЕНДАРНИЙ ПЛАН

№	Назва етапу виконання роботи	Тиждень семестру	Форма звітності
1	Отримання індивідуального завдання на курсову роботу	1–4	Затвердження теми (17.09.2026)
2	Розробка концептуальної моделі системи	4–5	Розділ 1 пояснювальної записки
3	Розробка формалізованої моделі системи	6–7	Розділ 2, модуль програмного забезпечення
4	Алгоритмізація моделі системи та її програмна реалізація	8–10	Модуль програмного забезпечення
5	Експериментальне дослідження моделі системи	11–12	Результати у вигляді тексту чи таблиць
6	Інтерпретація результатів моделювання, формулювання висновків та пропозицій	13	Розділ 5
7	Оформлення пояснювальної записки, подання на перевірку	14	Текст записки (не пізніше 11.11.2026)
8	Захист курсової роботи	15	Захист 18.11.2026

Студент _____ Андрій МЕЛЬНИЧУК

Керівник _____ Дмитро КОРИНЕЦЬКИЙ

АНОТАЦІЯ

Мельничук А. О. Курсова робота на тему «Імітаційна модель системи посадки літаків в аеропорті на основі формального опису мережею масового обслуговування».

Текстова частина курсової роботи складається із вступу, 5 розділів, висновків, списку використаних джерел з 16 найменувань та 2 додатків, які викладено на 80 сторінках. В тексті роботи оформлено 10 рисунків, 21 таблицю.

У роботі визначено кількість посадкових смуг аеропорту, за якої час окупності додаткових смуг T = S / p мінімальний, для пуассонівського потоку 10 літаків/год, дозаправки після 70±10 хв очікування та відходу в інший аеропорт після 140 хв очікування.

Модель побудовано на основі мережі масового обслуговування з універсальною багатоканальною СМО і чергою з обмеженим часом перебування. Алгоритм імітації реалізовано мовою C# і верифіковано за кожною вхідною змінною, протоколом подій і моделями M/M/m та M/M/m+M. Обґрунтовано перехідний період, тривалість і кількість прогонів; оптимум знайдено покроковим пошуком з перевіркою повним перебором із загальними випадковими числами і підтверджено статистичними критеріями.

З двома смугами аеропорт має збиток 42,5 млн грн/рік; мінімальний час окупності 0,0774 року (28,2 доби) досягається при 5 додаткових смугах. Оцінено вплив інтенсивності потоку і часу зайнятості смуги, зокрема повним факторним експериментом. Оптимум чутливий до інтенсивності потоку, тому рекомендовано побудувати п’ять додаткових смуг і зарезервувати майданчик ще під одну.

Ключові слова: ІМІТАЦІЙНА МОДЕЛЬ, МЕРЕЖА МАСОВОГО ОБСЛУГОВУВАННЯ, АЛГОРИТМ ІМІТАЦІЇ, ВЕРИФІКАЦІЯ, ПЛАНУВАННЯ ЕКСПЕРИМЕНТУ, ЧАС ОКУПНОСТІ.

ЗМІСТ

ВСТУП 7
1 КОНЦЕПТУАЛЬНА МОДЕЛЬ СИСТЕМИ 9
1.1 Опис системи та процесу посадки літаків 9
1.2 Вхідні змінні та параметри моделі 10
1.3 Вихідні змінні моделі 12
1.4 Мета моделювання та прийняті припущення 12
2 ФОРМАЛІЗОВАНА МОДЕЛЬ СИСТЕМИ 14
2.1 Елементи мережі масового обслуговування 14
2.2 Модифікація елемента «черга» для завдання В12 15
2.3 Формули обчислення вихідних характеристик 17
3 АЛГОРИТМІЗАЦІЯ ТА ПРОГРАМНА РЕАЛІЗАЦІЯ ІМІТАЦІЙНОЇ МОДЕЛІ СИСТЕМИ 19
3.1 Алгоритм імітації 19
3.1.1 Просування часу та зміна стану моделі 19
3.1.2 Збір статистики, перехідний період і складність 21
3.2 Вибір мови програмування та програмних засобів 22
3.3 Програмна реалізація 22
3.3.1 Структура класів 22
3.3.2 Ключові фрагменти коду 24
3.3.3 Запуск програми 25
3.4 Верифікація моделі 26
3.4.1 Перевірка при варіюванні вхідних змінних 26
3.4.2 Порівняння з аналітичними моделями 29
3.4.3 Перевірка протоколу подій 31
3.4.4 Висновок щодо коректності алгоритму 32
4 ЕКСПЕРИМЕНТАЛЬНЕ ДОСЛІДЖЕННЯ МОДЕЛІ 33
4.1 Мета використання моделі, фактор і відгук 33
4.2 Тривалість перехідного періоду 34

4.3 Тривалість прогону 35
4.4 Кількість прогонів 37
4.5 Серія експериментів повного перебору 38
4.6 Статистична обробка результатів 40
4.7 Програмне забезпечення для проведення експериментів 45
5 ІНТЕРПРЕТАЦІЯ РЕЗУЛЬТАТІВ МОДЕЛЮВАННЯ, ФОРМУЛЮВАННЯ ВИСНОВКІВ ТА ПРОПОЗИЦІЙ 46
5.1 Інтерпретація результатів і пропозиція 1 46
5.2 Пропозиція 2: скоротити час зайнятості смуги 46
5.3 Чутливість рекомендації до потоку і тлумачення прибутку 47
ВИСНОВКИ 49
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 51
ДОДАТОК А Лістинг коду алгоритму імітації 53
ДОДАТОК Б Лістинг коду проведення експериментів 67

ВСТУП

Тема курсової роботи – «Імітаційна модель системи посадки літаків в аеропорті на основі формального опису мережею масового обслуговування».

Аеропорт є типовою дискретно-подійною системою: його стан змінюється лише в моменти подій – надходження літака, звільнення смуги, досягнення порогу дозаправки і граничного часу очікування. За недостатньої кількості смуг утворюються черги в повітрі, літаки потребують дозаправки, частина з них відходить до інших аеропортів, і аеропорт втрачає дохід. Будівництво смуги потребує значних вкладень, тому рішення про кількість смуг має спиратися на кількісну оцінку окупності.

Аналітичні методи теорії масового обслуговування дають точні формули лише для обмеженого класу систем [1]. Детермінований час зайнятості смуги, випадкові пороги дозаправки, відхід з черги після фіксованого часу очікування і різний прибуток залежно від того, чи чекав літак, виводять задачу за межі класичних моделей. Для таких систем доцільно застосовувати імітаційне моделювання [2, 3, 4].

Метою роботи є визначення кількості посадкових смуг, за якої час окупності додаткових смуг $T = S/p$ буде мінімальним, де S – витрати на побудову смуг, p – прибуток за один рік. Об’єктом дослідження є процес посадки літаків в аеропорті з обмеженою кількістю смуг, предметом – імітаційна модель цього процесу на основі мережі масового обслуговування.

Для досягнення мети відповідно до навчального посібника [5] розв’язано такі завдання:

– розроблено концептуальну модель системи;

– виконано формалізацію мережею масового обслуговування з типом запиту і чергою з обмеженим часом перебування;

– розроблено і верифіковано власний алгоритм імітації;

– сплановано і проведено експериментальне дослідження моделі, отримано статистичні оцінки відгуку;

– інтерпретовано результати і сформульовано пропозиції щодо поліпшення роботи аеропорту.

Модель реалізовано мовою C# (.NET 10) з об’єктно-орієнтованим підходом до алгоритму імітації [6, 7], статистичну обробку – мовою Python з бібліотеками pandas, SciPy та matplotlib [8]. Пояснювальну записку оформлено відповідно до ДСТУ 3008:2015 [9], бібліографічний опис джерел – відповідно до ДСТУ 8302:2015 [10].

1 КОНЦЕПТУАЛЬНА МОДЕЛЬ СИСТЕМИ

1.1 Опис системи та процесу посадки літаків

В аеропорті є дві посадкові смуги, можна побудувати до десяти додаткових. Смуги ідентичні, кожна обслуговує один літак. Літаки надходять пуассонівським потоком з інтенсивністю 10 літаків/год (інтервал – експоненціальний із середнім 6 хв). Якщо є вільна смуга, літак сідає одразу і займає її 35 хв. Інакше він очікує в повітрі в черзі FIFO. Якщо очікування триває довше за 70±10 хв, потрібна дозаправка, на яку аеропорт витрачає 1000±200 грн (грошові одиниці умови прийнято гривнями). Після 140 хв очікування літак відходить в інший аеропорт: прибутку немає, витрати на дозаправку залишаються. За посадку без очікування аеропорт отримує 2000 грн, після очікування – 1500±100 грн. Додаткова смуга коштує 3 000 000 грн.

Середня кількість смуг, потрібна для потоку, дорівнює $\lambda t_D = \frac{1}{6} \cdot 35 \approx 5{,}83$, тому навантаження $\rho = 5{,}83/m$: за $m \leq 5$ система перевантажена, і лише з $m = 6$ пропускна здатність перевищує потік.

Процес подано діаграмою BPMN [5, 11] (рис. 1.1). Множина подій системи: 1) надходження літака; 2) звільнення смуги через 35 хв після посадки; 3) потреба в дозаправці, коли очікування досягає порогу 70±10 хв; 4) відхід в інший аеропорт, коли очікування досягає 140 хв. Події 1 і 2 – це події стандартних елементів мережі МО (надходження і вихід запиту з каналу), події 3 і 4 відбуваються з літаками в черзі. Посадка не є окремою подією: вона починається в момент надходження (за вільної смуги, умова – є вільна смуга) або в момент звільнення смуги (для першого літака черги, умова – черга непорожня). Подія 3 настає, якщо літак ще в черзі і не дозаправлявся, подія 4 – якщо літак ще в черзі.

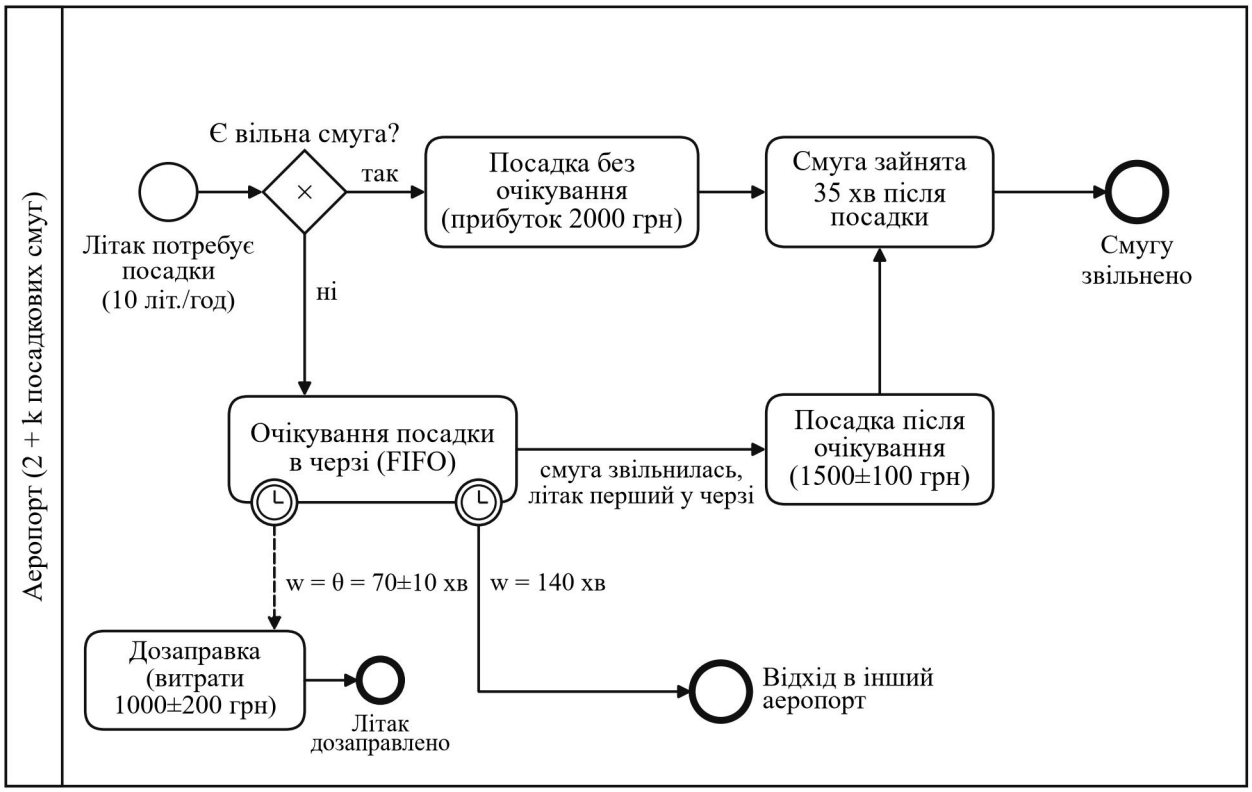


Рисунок 1.1 – Схема процесу посадки літака в нотації BPMN

1.2 Вхідні змінні та параметри моделі

Вхідні змінні – величини, вплив яких на результат досліджується [5] (табл. 1.1). До них віднесено всі величини з умови завдання, що можуть змінюватися: кожна з них варіюється щонайменше під час верифікації (підрозділ 3.4). Основна вхідна змінна – кількість додаткових смуг k; інтенсивність потоку λ і час зайнятості смуги t_D варіюються також в експериментальному дослідженні для оцінки стійкості рішення. Позначення unif(d, s) відповідає рівномірному розподілу на інтервалі $(d - s; d + s)$ [5], тому «70±10», «1000±200» і «1500±100» трактуються як unif(70, 10), unif(1000, 200) і unif(1500, 100).

Таблиця 1.1 – Вхідні змінні моделі

Вхідна змінна, позначення	**Значення за умовою**	**Діапазон варіювання**	**Закон розподілу**	**Одиниця виміру**
Кількість додаткових посадкових смуг k (усього смуг $m = 2 + k$)	0	{0, 1, …, 10}	детермінована	шт.

Продовження таблиці 1.1

Вхідна змінна, позначення	**Значення за умовою**	**Діапазон варіювання**	**Закон розподілу**	**Одиниця виміру**
Інтенсивність потоку літаків λ (середній інтервал $t_C = 60/\lambda$ хв)	10	8…12 у дослідженні; 2…12 у верифікації	пуассонівський потік	літ./год
Час зайнятості смуги після посадки t_D	35	30…35 у дослідженні; 20 у верифікації	детермінований	хв
Поріг очікування для дозаправки θ_i (окремо для кожного літака)	unif(70, 10)	unif(100, 10), без дозаправки – у верифікації	рівномірний	хв
Витрати аеропорту на одну дозаправку c_i	unif(1000, 200)	unif(2000, 200) у верифікації	рівномірний	грн
Максимальний час очікування посадки $w_{\max}$	140	200, без відходу – у верифікації	детермінований	хв
Прибуток за посадку без очікування π_0	2000	4000 у верифікації	детермінований	грн
Прибуток за посадку після очікування π_i	unif(1500, 100)	unif(3000, 100) у верифікації	рівномірний	грн
Вартість будівництва однієї додаткової смуги s	3 000 000	в імітацію не входить, змінює T пропорційно	детермінована	грн

Сталі параметри моделі наведено в таблиці 1.2. Модельний час вимірюється в хвилинах, $\lambda = 1/6$ літ./хв.

Таблиця 1.2 – Сталі параметри моделі

Параметр	**Позначення**	**Значення**	**Одиниця виміру**
Кількість наявних посадкових смуг	m_0	2	шт.
Найбільша кількість додаткових смуг	–	10	шт.
Тривалість одного року	–	525 600	хв

1.3 Вихідні змінні моделі

Основна вихідна змінна – час окупності додаткових смуг; для нього потрібен річний прибуток, що визначається кількістю посадок кожного типу і витратами на дозаправку. Показники черги, очікування, відходів і завантаження смуг пояснюють результати і використовуються у верифікації (табл. 1.3).

Таблиця 1.3 – Вихідні змінні моделі

Вихідна змінна, позначення	**Спостереження під час імітації**	**Одиниця виміру**
Кількість надходжень N_C, посадок без очікування N_0 і після очікування N_w, дозаправок N_r, відходів N_d	лічильники відповідних змін стану	шт.
Середня і максимальна довжина черги $\overline{L}$, $L_{\max}$	довжина черги з вагою тривалості інтервалу між змінами стану	літаків
Середнє завантаження смуги $\overline{\rho}$	кількість зайнятих смуг з вагою тривалості інтервалу, поділена на m	–
Середній час очікування $\overline{w}$ (допоміжна – $\overline{w}_w$)	сумарне очікування, поділене на кількість усіх літаків, що пройшли через СМО; $\overline{w}_w$ – лише для тих, що сіли після очікування	хв
Частка відходів P_d, дозаправок P_r	відношення кількостей до N_C	–
Річний прибуток $p(m)$	дохід від посадок мінус витрати на дозаправку за рік	грн/рік
Час окупності $T(k)$	$S(k)/\Delta p(k)$	років (діб)

1.4 Мета моделювання та прийняті припущення

Мета моделювання – визначити кількість смуг, за якої час окупності додаткових смуг $T = S/p$ мінімальний. Умова не уточнює, що таке p; у роботі це приріст річного прибутку завдяки новим смугам: $\Delta p(k) = p(2+k) - p(2)$. Витрати S стосуються лише нових смуг, тому й прибуток має бути тим, який вони приносять; крім того, повний прибуток аеропорту за малих k від'ємний. Стійкість висновку до іншого тлумачення перевірено в підрозділі 5.3. Отже, потрібно знайти $k^* \in \{1, \ldots, 10\}$ з мінімальним $T(k)$.

Прийнято такі припущення:

– рік – 525 600 хв; річний прибуток – перерахунок прибутку за період збору статистики;

– на початку всі смуги вільні, черга порожня, дані перехідного періоду не враховуються;

– дозаправка – лише витрати аеропорту в момент досягнення порогу; літак не втрачає місця в черзі й дозаправляється не більше одного разу (поріг не перевищує 80 хв);

– витрати на дозаправку несе аеропорт і для літаків, які потім відійшли;

– прибуток зараховується в момент, коли літак займає смугу;

– за одночасних змін стану першим обробляється звільнення смуги, останнім – надходження нового літака.

2 ФОРМАЛІЗОВАНА МОДЕЛЬ СИСТЕМИ

2.1 Елементи мережі масового обслуговування

Формалізацію виконано мережею масового обслуговування (МО), елементи і позначення якої визначено в таблиці 3.1 навчального посібника [5]: d – детерміноване значення, $\exp(d)$ – експоненціальний закон із середнім d, $\mathrm{unif}(d, s)$ – рівномірний закон на $(d - s; d + s)$.

Запит мережі – літак. Ідентичні смуги є пристроями однієї багатоканальної СМО зі спільною чергою (рис. 2.1): надходження С з інтервалом $\exp(6)$ хв, черга Q, $m = 2 + k$ пристроїв $D_1, ..., D_m$ з часом обслуговування 35 хв, вихід N_1 «посадку завершено» і вихід N_2 «відхід в інший аеропорт», до якого запит потрапляє з черги. Числові параметри елементів наведено в таблиці 2.1. Тип запиту τ_i – атрибут запиту, а не елемент мережі; він визначає прибуток за посадку (підрозділ 2.2).

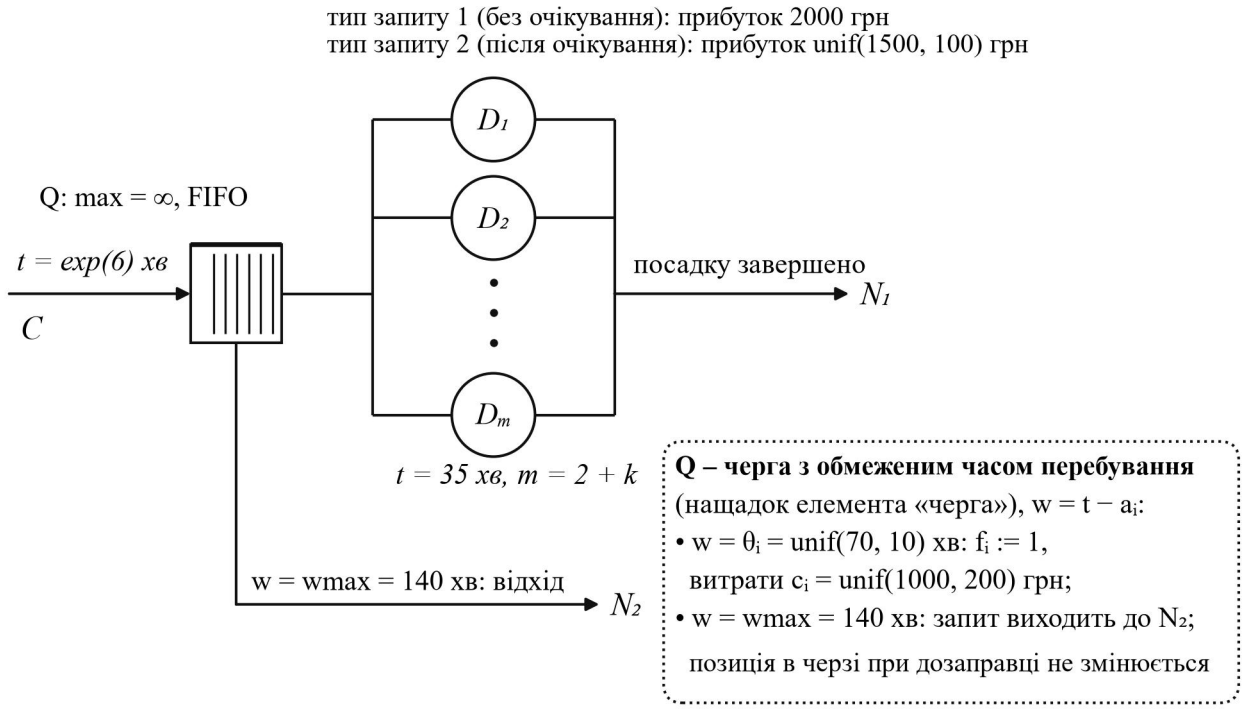


Рисунок 2.1 – Мережа масового обслуговування, що формально представляє процес посадки літаків в аеропорті

Таблиця 2.1 – Елементи формалізованої моделі та їх числові параметри

Елемент мережі МО	**Позначення**	**Числові параметри**	**Зміст**
Надходження запитів ззовні	C	$t = \exp(6)$ хв	літак отримує атрибути a_i, θ_i, c_i, π_i
Черга з обмеженим часом перебування	Q	max = ∞; FIFO; $w_{\max} = 140$ хв; $\theta_i = \text{unif}(70{,}10)$ хв; $c_i = \text{unif}(1000{,}200)$ грн	при $w_i = \theta_i$ – ознака $f_i := 1$ і витрати c_i; при $w_i = w_{\max}$ – вихід до N_2
Пристрої СМО	$D_1, \ldots, D_m$	$m = 2 + k, k = 0 \ldots 10$; $t = 35$ хв	посадкові смуги
Виходи	N_1, N_2	–	посадку завершено; відхід в інший аеропорт

2.2 Модифікація елемента «черга» для завдання B12

Стандартні елементи мережі МО не відтворюють дозаправки під час очікування, відходу з черги після 140 хв і залежності прибутку від очікування. Посібник допускає ускладнення параметрів елементів і врахування типу замовлення [5], а специфіку задачі в курсі рекомендовано реалізовувати класом-нащадком стандартного елемента, не змінюючи універсального алгоритму [7]. Тому модель має чотири події (таблиця 2.2): дві події стандартних елементів – надходження запиту і вихід запиту з каналу – і дві події модифікованого елемента «черга з обмеженим часом перебування» – дозаправку в момент $a_i + \theta_i$ і вихід з черги за часом очікування в момент $a_i + w_{\max}$. Прибуток задано типом запиту. Найближчий момент подій черги t_Q входить до моменту найближчої зміни стану мережі нарівні з моментами інших елементів.

Атрибути запиту: a_i – момент надходження; θ_i – поріг очікування для дозаправки; c_i – витрати на дозаправку; π_i – прибуток за посадку після очікування; $f_i = 0$ – ознака дозаправки; $w_i(t) = t - a_i$ – поточний час очікування.

Тип запиту τ_i – атрибут запиту, що визначається в момент заняття каналу (у програмі – ознака `waited` події `ChannelOccupied`) і визначає прибуток, а

не час обслуговування: якщо літак займає вільну смугу в момент надходження, $\tau_i = 1$ і прибуток $\pi_0 = 2000$ грн; якщо після черги, $\tau_i = 2$ і прибуток π_i.

Черга з обмеженим часом перебування – черга FIFO, у якій у момент $a_i + \theta_i$ запит отримує ознаку $f_i := 1$, а аеропорт – витрати c_i (позиція в черзі не змінюється), а в момент $a_i + w_{\max}$ запит залишає чергу до виходу N_2. Оскільки $\theta_i < 80 < w_{\max}$, дозаправка буває не більше одного разу; якщо запит займає смугу раніше $a_i + \theta_i$, її немає.

Порядок одночасних змін стану: вихід з каналу (канал займає перший запит черги), зміна стану черги, надходження. Тоді літак, для якого звільнилася смуга, сідає, а не дозаправляється чи відходить, а новий літак займає смугу лише за порожньої черги.

Стан моделі S задається моментом наступного надходження t_0, станами пристроїв $K_r \in \{0,1\}$ з моментами виходу з каналу t_r ($t_r = \infty$ для вільного) і впорядкованою чергою довжиною L; $K = \sum_{r=1}^{m} K_r$. Момент найближчої зміни стану мережі визначається за формулою:

$$t_{\text{next}} = min\big(t_0,\ t_1, \ldots, t_m,\ t_Q\big), \tag{2.1}$$

де t_Q – момент найближчої зміни стану черги, що визначається за формулою:

$$t_Q = min\big(min_{i \in Q,\, f_i = 0}(a_i + \theta_i),\ min_{i \in Q}(a_i + w_{\max})\big), \tag{2.2}$$

де Q – множина запитів у черзі. Зміну стану елементів наведено в таблиці 2.2 (g_C – згенерований інтервал надходження).

Таблиця 2.2 – Зміна стану елементів мережі МО в момент t

Подія	**Умова**	**Зміна стану елементів**	**Зміна накопичуваних величин**
Надходження запиту (С)	$t_0 = t$	$t_0 := t + g_C$; запит i: $a_i := t$, генеруються θ_i, c_i, π_i, $f_i := 0$. Якщо існує r: $K_r = 0$, то $K_r := 1$, $t_r := t + 35$, $\tau_i := 1$; інакше $L := L + 1$	$N_C := N_C + 1$; при $\tau_i = 1$: $N_0 := N_0 + 1$, $P := P + \pi_0$
Вихід запиту з каналу D_r	$t_r = t$	$K_r := 0$, $t_r := \infty$, запит – у N_1. Якщо $L > 0$: перший запит j черги, $L := L - 1$, $K_r := 1$, $t_r := t + 35$, $\tau_j := 2$	$N_1 := N_1 + 1$; якщо $L > 0$: $N_w := N_w + 1$, $P := P + \pi_j$, $\Sigma w := \Sigma w + (t - a_j)$
Черга Q: дозаправка	$a_i + \theta_i = t$, $f_i = 0$	$f_i := 1$	$N_r := N_r + 1$, $P := P - c_i$
Черга Q: вихід за часом очікування	$a_i + w_{\max} = t$	запит i – у N_2, $L := L - 1$	$N_d := N_d + 1$, $\Sigma w := \Sigma w + w_{\max}$

2.3 Формули обчислення вихідних характеристик

Випадкові величини генеруються методом оберненої функції з рівномірного на $(0; 1)$ числа u [2] за формулами:

$$x = -d \cdot \ln u, x = (d - s) + 2su, \tag{2.3}$$

де перша формула відповідає закону $\exp(d)$, друга – закону $\text{unif}(d, s)$.

Статистика збирається на інтервалі $[T_{\text{warm}}; T_{\text{mod}}]$ тривалістю $T_{\text{ст}} = T_{\text{mod}} - T_{\text{warm}}$ (обґрунтовані в розділі 4 значення: $T_{\text{warm}} = 3000$ хв, $T_{\text{ст}} = 525600$ хв, $T_{\text{mod}} = 528600$ хв). Час просувається на нерівні інтервали Δt_j, тому середня довжина черги і середнє завантаження смуги обчислюються за формулами [5]:

$$\overline{L} = \frac{1}{T_{\text{ст}}} \sum_j L_j \, \Delta t_j, \overline{\rho} = \frac{1}{mT_{\text{ст}}} \sum_j K_j \, \Delta t_j, \tag{2.4}$$

де L_j, K_j – довжина черги і кількість зайнятих смуг на j-му інтервалі; $L_{\max} = max_j L_j$.

Середній час очікування обчислюється за всіма літаками, що пройшли через СМО, зокрема тими, що не чекали або відійшли; допоміжна характеристика – очікування лише тих, хто сів після черги:

$$\overline{w} = \frac{\Sigma w}{N_0+N_w+N_d}, \overline{w}_w = \frac{1}{N_w}\sum_{i\in I_w}\left(t_i^{\text{пос}} - a_i\right), \tag{2.5}$$

де I_w – множина літаків, що сіли після очікування, $t_i^{\text{пос}}$ – момент посадки. Частки відходів і дозаправок: $P_d = N_d/N_C$, $P_r = N_r/N_C$.

Річний прибуток аеропорту з m смугами обчислюється за формулою:

$$p(m) = \left(\pi_0 N_0 + \sum_{i\in I_w}\pi_i - \sum_{i\in I_r} c_i\right)\frac{525600}{T_{\text{ст}}}, \tag{2.6}$$

де I_r – множина дозаправлених літаків (зокрема тих, що потім відійшли). Приріст прибутку, витрати і час окупності (у роках) визначаються за формулами:

$$\Delta p(k) = p(2+k) - p(2), S(k) = 3{\cdot}10^6{\cdot}k, T(k) = \frac{S(k)}{\Delta p(k)} \tag{2.7}$$

а шукана кількість додаткових смуг – умовою:

$$k^* = \underset{k=1,\ldots,10}{\operatorname{argmin}} T(k) \tag{2.8}$$

У прогоні з номером l усі значення k використовують однакові послідовності випадкових чисел (метод загальних випадкових чисел), тому Δp і T обчислюються в межах прогону, а оцінкою є середнє за n прогонами:

$$\overline{T}(k) = \frac{1}{n}\sum_{l=1}^{n}\frac{S(k)}{p_l(2+k) - p_l(2)} \tag{2.9}$$

Середнє відношень (2.9) дає змогу обчислити довірчий інтервал за вибіркою $T_l(k)$; від відношення $S(k)$ до середнього за прогонами приросту $\Delta p(k)$ воно відрізняється не більше ніж на $1{,}3{\cdot}10^{-5}$ року (≈ 0,005 доби) для всіх k, бо розкид Δp малий порівняно із середнім.

Вимоги до точності: $T(k)$ – з точністю $\varepsilon_T = 0{,}0005$ року (≈ 0,18 доби), $p(m)$ – з точністю $\varepsilon_p = 1$ млн грн/рік, обидві з довірчою ймовірністю $\beta = 0{,}95$.

3 АЛГОРИТМІЗАЦІЯ ТА ПРОГРАМНА РЕАЛІЗАЦІЯ ІМІТАЦІЙНОЇ МОДЕЛІ СИСТЕМИ

3.1 Алгоритм імітації

3.1.1 Просування часу та зміна стану моделі

Алгоритм імітації розв’язує три задачі: просування модельного часу, зміну стану в моменти подій і збір статистики [2, 5]. Використано універсальний алгоритм імітації мережі МО з просуванням часу до найближчої події: визначити t_{next} за формулою (2.1), накопичити статистику на $[t; t_{\text{next}}]$, перенести час $t := t_{\text{next}}$ і змінити стан елементів, у яких момент найближчої зміни стану дорівнює t. Фіксований крок не використано: експоненціальні інтервали не мають найменшого значення, і крок дав би або холості кроки, або неточні моменти подій [5].

Події стандартних елементів – надходження (С) і вихід запиту з каналу (D_r) – обробляють універсальні елементи «надходження» і «СМО», а події дозаправки і виходу з черги в моменти $a_i + \theta_i$ і $a_i + w_{\max}$ – модифікований елемент «черга» (підрозділ 2.2, таблиця 2.2). Блок-схему наведено на рисунку 3.1, псевдокод – у лістингах 3.1 і 3.2 (у позначеннях алгоритмів 3.1 і 3.2 посібника [5]).

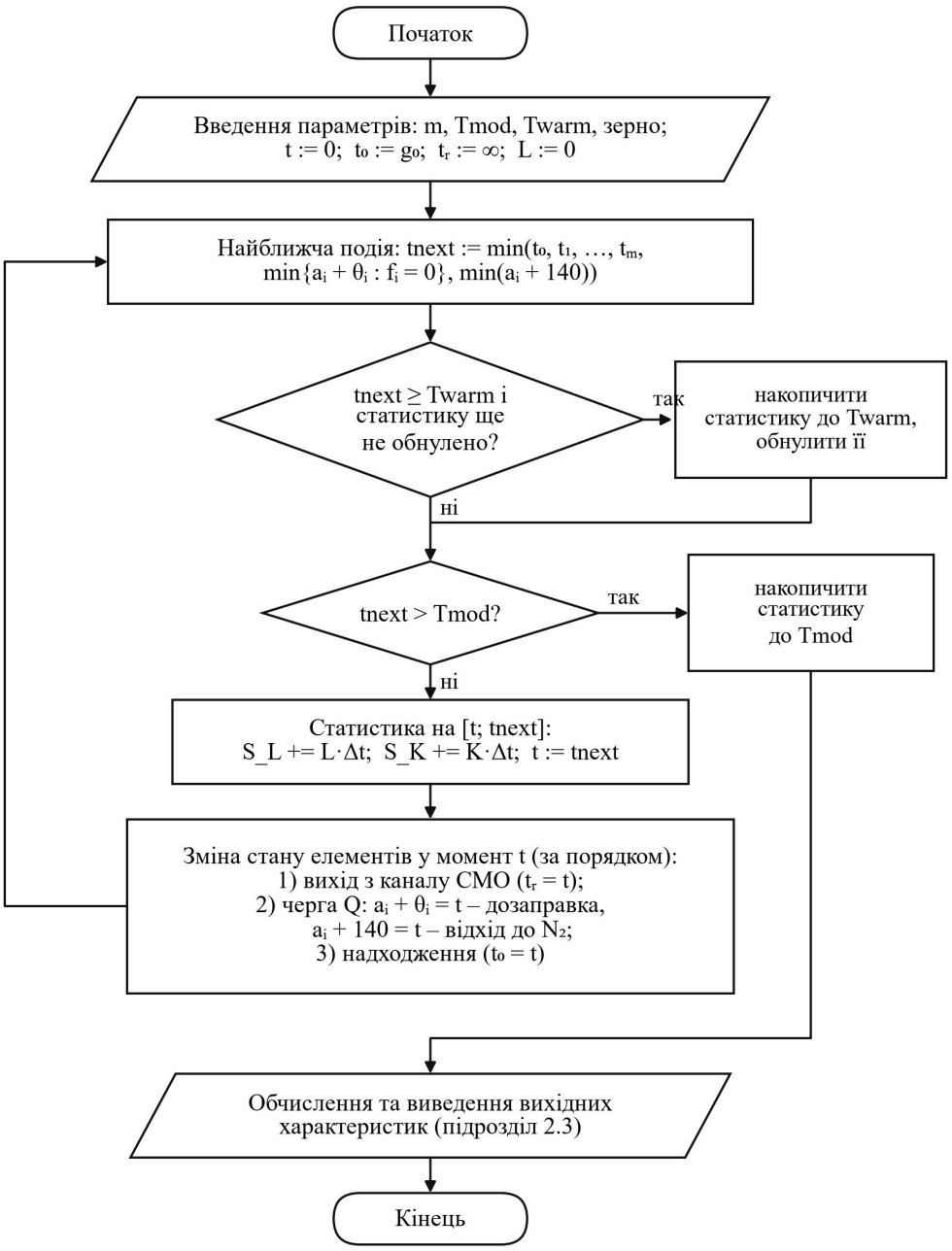


Рисунок 3.1 – Блок-схема алгоритму імітації мережі МО посадки літаків

Лістинг 3.1 – Псевдокод основного циклу імітації

```
Алгоритм 3.1. Імітація мережі МО посадки літаків
вхід: m, Tmod, Twarm, зерно; вхідні змінні і параметри (таблиці 1.1, 1.2)
  t := 0;  t0 := g0 ~ exp(6);  tr := ∞, Kr := 0 (r = 1..m);  Q := порожня
  обнулено := (Twarm ≤ 0)
  поки істина
    tnext := min(t0, min tr, tQ)                    // формули (2.1), (2.2)
    якщо не обнулено і tnext ≥ Twarm то            // відкинути перехідний
    // період
      SL += L·(Twarm − t);  SK += K·(Twarm − t);  t := Twarm
      обнулити лічильники і суми;  обнулено := істина
    якщо tnext > Tmod то
      SL += L·(Tmod − t);  SK += K·(Tmod − t);  t := Tmod
      вийти з циклу
    SL += L·(tnext − t);  SK += K·(tnext − t)     // збір статистики
    t := tnext                                     // просування часу
    S := E(S, t)                                   // алгоритм 3.2
```

```
  обчислити й вивести вихідні характеристики (підрозділ 2.3)
```

Лістинг 3.2 – Псевдокод перетворення стану E(S, t)

```
Алгоритм 3.2. Перетворення стану E(S, t)
  для r = 1..m                                  // 1) вихід запиту з каналу СМО
    якщо tr = t то
      Kr := 0;  tr := ∞;  N1 += 1
      якщо L > 0 то                             // канал займає перший запит черги
        i := перший(Q);  вилучити i з Q;  L −= 1;  Σw += t − ai
        Kr := 1;  tr := t + 35;  Nw += 1;  Дохід += πi
  якщо tQ = t то                                // 2) зміна стану черги Q
    для кожного i з Q
      якщо fi = 0 і ai + θi = t то
        fi := 1;  Nr += 1;  Витрати += ci
    для кожного i з Q
      якщо t − ai ≥ wmax то
        вилучити i з Q;  L −= 1;  Nd += 1;  Σw += wmax
  якщо t0 = t то                                // 3) надходження запиту
    створити літак i: ai := t, θi ~ unif(70, 10),
                      ci ~ unif(1000, 200), πi ~ unif(1500, 100), fi := 0
    t0 := t + g0;  NC += 1
    якщо існує r: Kr = 0 то
      Kr := 1;  tr := t + 35;  N0 += 1;  Дохід += 2000
    інакше
      додати i в кінець Q;  L += 1
  повернути S
```

3.1.2 Збір статистики, перехідний період і складність

Довжина черги і кількість зайнятих смуг накопичуються з вагою інтервалу між змінами стану (формула (2.4)), лічильники, дохід, витрати і сумарне очікування – у моменти змін стану. Модель стартує з порожньої системи; у момент T_{warm} лічильники й суми обнуляються методом `ResetStatistics()`, а стан моделі зберігається.

Порядок одночасних змін стану задано логікою процесу (підрозділ 2.2), тому випадковий вибір порядку, який посібник [5] радить за відсутності такого правила, не потрібен; у програмі його задає порядок елементів у списку моделі. Пошук мінімуму серед п’яти елементів виконується за сталий час, перерахунок моменту СМО і черги – за $O(m + L)$; черга обмежена відходами: у ній перебувають лише літаки, що надійшли за останні $w_{\max} = 140$ хв, тож у середньому $\overline{L} \le \lambda w_{\max} \approx 23$, а найбільша довжина за рік не перевищує 53 (таблиця 4.6). Річний прогін (176–264 тис. кроків) триває 0,1–0,4 с, тож календар подій не потрібен.

3.2 Вибір мови програмування та програмних засобів

Модель реалізовано мовою C# (.NET 10) [6] на основі об’єктно-орієнтованого підходу до алгоритму імітації з дистанційного курсу [7]: абстрактний елемент мережі МО і його нащадки. Компільований код виконує річний прогін за частки секунди, PLINQ – реплікації паралельно, а генератор `System.Random` із заданим зерном робить прогони відтворюваними. Статистичну обробку виконано мовою Python (pandas, SciPy [8], matplotlib). Готових засобів імітації (Arena [12], CPN Tools, SimPy) не використано: алгоритм власний [5]. Код розміщено в репозиторії https://github.com/QuadRaliOn/kmdps-coursework-b12-airport-landing, лістинги – у додатках А і Б.

3.3 Програмна реалізація

3.3.1 Структура класів

Діаграму класів наведено на рисунку 3.2, призначення класів – у таблиці 3.1. Елементи мережі МО успадковують абстрактний клас `Element` (поточний час, момент найближчої зміни стану `Tnext`, кількість запитів, що пройшли через елемент, методи `InAct()`, `OutAct()`, `DoStatistics()`). Кожен тип елемента формалізму реалізовано одним класом, специфіку задачі – параметрами, класом-нащадком черги і підписками на події елементів.

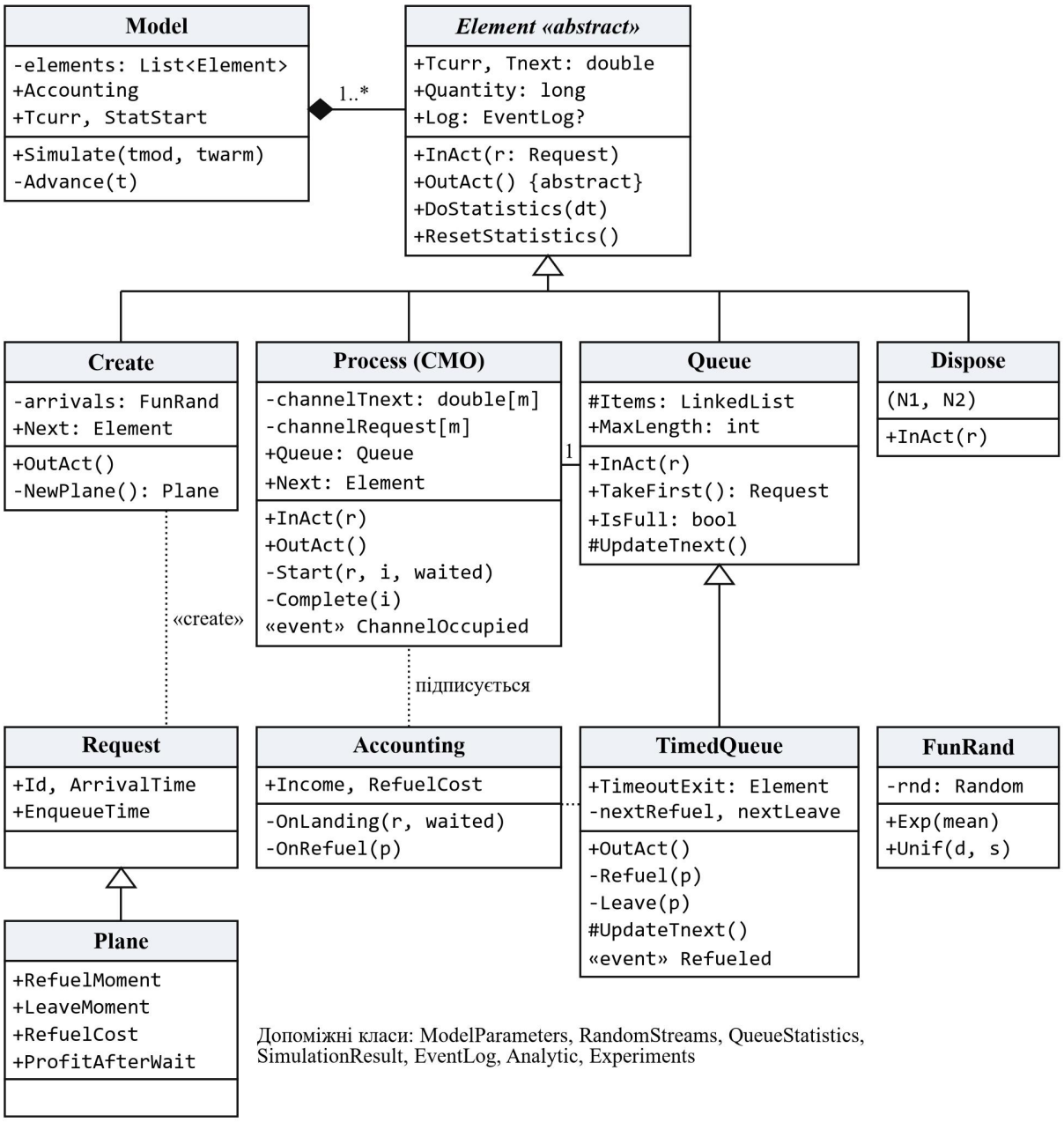


Рисунок 3.2 – Діаграма класів програми AirportSim

Таблиця 3.1 – Класи програми та їх призначення

Клас	Призначення
`Element`	абстрактний елемент мережі МО
`Create`	надходження: генерує запити з інтервалом exp(6) хв і їхні атрибути
`Process`	універсальна багатоканальна СМО: m каналів, час обслуговування – параметр, черга; подія – вихід запиту з каналу; відмова за заповненої черги
`Queue`	універсальна черга FIFO з максимальною довжиною `MaxLength`
`TimedQueue`	нащадок `Queue`: дозаправка при $w_i = \theta_i$, вихід до N_2 при $w_i = w_{\max}$
`Dispose`	виходи N_1 і N_2

Продовження таблиці 3.1

Клас	Призначення	
`Model`	мережа МО задачі B12 і основний цикл `Simulate()`	
`Request`, `Plane`	запит і його нащадок – літак з атрибутами θ_i, c_i, π_i, f_i	
`Accounting`	облік доходу за типом запиту і витрат на дозаправку	
`ModelParameters`, `FunRand`, `RandomStreams`	параметри; генератори exp(d), unif(d, s) і три потоки випадкових чисел	
`QueueStatistics`, `SimulationResult`, `EventLog`	накопичувані суми, вихідні характеристики, протокол подій	
`Analytic`, `Experiments`	формули Ерланга C і A, верифікація та серії експериментів	

3.3.2 Ключові фрагменти коду

Метод `Process.InAct()` ставить запит у вільний канал з найменшим номером, за зайнятих каналів – у чергу, а за заповненої черги фіксує відмову (наприклад, за `--maxqueue 5` і $m = 2$ модель дає 57 159 відмов за рік). Метод `Process.OutAct()` (лістинг 3.3) обробляє єдину подію СМО – вихід запиту з каналу. Час обслуговування задає параметр `serviceTime`, а заняття каналу повідомляється подією `ChannelOccupied`, на яку підписаний клас `Accounting`: він додає до доходу π_0 або π_i залежно від того, чи чекав літак.

Лістинг 3.3 – Метод Process.OutAct() – вихід запитів з каналів СМО в момент t

```
public override void OutAct()
{
    for (int r = 0; r < channelTnext.Length; r++)
    {
        if (channelTnext[r] > Tcurr) continue;
        Complete(r);
        Queue.Tcurr = Tcurr;
        if (Queue.TakeFirst() is { } next)
            Start(next, r, waited: true);
    }
    UpdateTnext();
}
```

Клас `TimedQueue` – нащадок універсальної черги, як рекомендовано для черг із додатковими умовами залишення [7]. Його метод `OutAct()` (лістинг 3.4) виконується в момент t_Q: запити, що чекали θ_i, отримують ознаку дозаправки, а запити, що чекали $w_{\max}$, виходять до N_2. Моменти $a_i + \theta_i$ і $a_i + w_{\max}$ обчислюються один раз при створенні літака, тому порівняння виконуються без похибки округлення.

Лістинг 3.4 – Метод TimedQueue.OutAct() – зміна стану черги з обмеженим часом перебування

```
public override void OutAct()
{
    if (refuelEnabled && nextRefuel <= Tcurr)
        foreach (Request request in Items)
            if (request is Plane { Refueled: false } plane &&
                plane.RefuelMoment <= Tcurr)
                Refuel(plane);
    if (nextLeave <= Tcurr)
    {
        LinkedListNode<Request>? node = Items.First;
        while (node != null)
        {
            LinkedListNode<Request>? next = node.Next;
            if (node.Value is Plane plane && plane.LeaveMoment <= Tcurr)
            {
                Items.Remove(node);
                Leave(plane);
            }
            node = next;
        }
    }
    UpdateTnext();
}
```

Елемент `Create` генерує всі атрибути літака в момент надходження з окремого потоку випадкових чисел, тому моделі з різною кількістю смуг за однакового зерна отримують однакову послідовність літаків (метод загальних випадкових чисел, підрозділ 4.1). Основний цикл `Model.Simulate()` (додаток А) відповідає лістингу 3.1.

3.3.3 Запуск програми

AirportSim – консольний застосунок; режим задає команда (`run`, `trace`, `verify`, `transient`, `series`), а вхідні змінні – параметри командного рядка (`--m`, `--tmod`, `--twarm`, `--seed`, `--interarrival`, `--occupancy`, `--maxwait`, `--maxqueue` тощо), тож модель запускається зі зміненими параметрами без

зміни коду. Результат прогону для $m = 7$ за рік збору статистики показано на рисунку 3.3: надійшло 87 241 літак, 41 047 сіли без очікування, 46 194 – після очікування, дозаправок 430, відходів немає, річний прибуток – 150,96 млн грн.

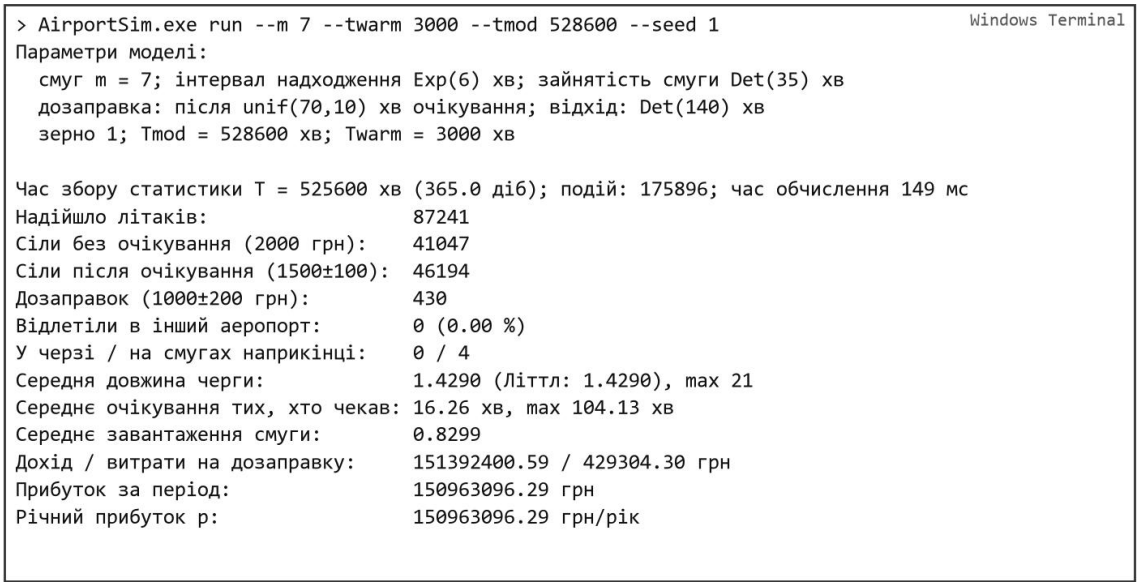


Рисунок 3.3 – Результат запуску програми для m = 7 смуг (час моделювання 528 600 хв, перехідний період 3000 хв, зерно 1)

3.4 Верифікація моделі

Верифікація має довести, що алгоритм відтворює функціонування системи відповідно до формалізованої моделі [5]. Виконано три перевірки: прогони при зміні вхідних змінних і параметрів, порівняння з аналітичними моделями і ручну перевірку протоколу подій.

3.4.1 Перевірка при варіюванні вхідних змінних

Результати 20 прогонів тривалістю 525 600 хв (зерно 1, без відкидання перехідного періоду, щоб перевірити баланс літаків) наведено в таблиці 3.2. Не зазначені в назві варіанта значення дорівнюють умові завдання. Кожен рядок відрізняється від свого базового варіанта однією зміною: рядки «m = 3 … 12» і «m = 2, …» – від варіанта $m = 2$, рядки «m = 7, …» – від рядка $m = 7$, рядки «m = 6, …» – від рядка $m = 6$ (там система не перевантажена, і вплив дозаправки, відходу та потоку помітніший).

Таблиця 3.2 – Результати верифікаційних прогонів (час моделювання 525 600 хв, зерно 1)

Варіант	N_C	N_0	N_w	N_r	N_d	$\overline{L}$	$\overline{\rho}$	$\overline{w}$, **хв**	**р, млн грн/рік**
базовий, m = 2	87 198	2	30 032	87 182	57 143	22,772	1,0000	137,28	−42,090
m = 3	87 198	3	45 048	87 161	42 126	22,370	1,0000	134,86	−19,549

Продовження таблиці 3.2

Варіант	N_C	N_0	N_w	N_r	N_d	$\overline{L}$	$\overline{\rho}$	$\overline{w}$, **хв**	**р, млн грн/рік**
m = 4	87 198	4	60 064	86 998	27 111	21,696	1,0000	130,79	3,139
m = 5	87 198	136	74 914	82 953	12 132	19,739	0,9995	118,99	29,725
m = 6	87 198	10 555	75 824	24 737	806	7,906	0,9586	47,65	110,100
m = 7	87 198	41 176	46 022	411	0	1,409	0,8295	8,50	150,981
m = 12	87 198	85 810	1 388	0	0	0,012	0,4839	0,07	173,703
m = 7, інтервал exp(5)	104 906	6 404	96 737	47 615	1 745	12,952	0,9811	64,89	110,310
m = 7, інтервал exp(7,5)	69 993	53 707	16 286	0	0	0,270	0,6658	2,03	131,851
m = 2, інтервал exp(12)	43 699	14	30 017	42 354	13 658	10,338	0,9998	124,36	2,726
m = 2, інтервал exp(30)	17 404	10 197	7 207	126	0	0,296	0,5794	8,94	31,082
m = 2, смуга 20 хв	87 198	2	52 558	87 140	34 619	22,246	1,0000	134,11	−8,255
m = 2, $w_{\max}$ = 200 хв	87 198	2	30 032	87 183	57 138	32,724	1,0000	197,28	−42,074
m = 2, інтервал 6 хв (детерм.)	87 600	2	30 032	87 581	57 542	23,185	1,0000	139,13	−42,483
m = 6, θ = unif(100, 10)	87 198	10 555	75 824	12 739	806	7,906	0,9586	47,65	122,110
m = 6, без дозаправки	87 198	10 555	75 824	0	806	7,906	0,9586	47,65	134,850
m = 6, без відходу	87 198	8 380	78 798	37 258	0	14,092	0,9675	84,94	97,697
m = 6, c = unif(2000, 200)	87 198	10 555	75 824	24 737	806	7,906	0,9586	47,65	85,363
m = 6, π_0 = 4000 грн	87 198	10 555	75 824	24 737	806	7,906	0,9586	47,65	131,210
m = 6, π = unif(3000, 100)	87 198	10 555	75 824	24 737	806	7,906	0,9586	47,65	223,836

Поведінка моделі відповідає логіці системи:

1) баланс $N_C = N_0 + N_w + N_d + L_{\text{кінц}}$ виконується точно в усіх прогонах ($L_{\text{кінц}}$ – довжина черги наприкінці, для базового варіанта $2 + 30032 + 57143 + 21 = 87198$);

2) за $m \le 4$ смуги зайняті весь час, і посадок рівно стільки, скільки дозволяє пропускна здатність $m{\cdot}525600/35$ (30 034, 45 051, 60 068); за $m = 5$ смуги зайняті 99,95 % часу, і посадок (75 050) лише на 35 менше за пропускну

здатність 75 085, бо черга зрідка спорожнюється; кожна смуга зменшує кількість відходів приблизно на 15 017, майже всі літаки дозаправляються, прибуток від'ємний;

3) за $m \geq 6$ завантаження збігається з теоретичним $(N_0 + N_w)t_D/(mT_{\text{mod}})$ (для $m = 7$: $87198{\cdot}35/(7{\cdot}525600) = 0{,}8295$), за інтервалу exp(30) – з $35/(30{\cdot}2) = 0{,}583$, за exp(7,5) – з $35/(7{,}5{\cdot}7) = 0{,}667$;

4) зайнятість смуги 20 хв дає рівно 52 560 посадок, $w_{\max} = 200$ хв подовжує чергу без зміни кількості посадок, без відходу черга і очікування зростають, а за детермінованих надходжень максимальне очікування дорівнює точно 140 хв завдяки порядку змін стану (п. 3.4.3);

5) поріг і вартість дозаправки та прибутки за посадку за $m = 6$ змінюють лише N_r і прибуток, причому точно на очікувану величину: подвоєння вартості дозаправки зменшує прибуток на 24,737 млн грн (24 737 дозаправок по 1000 грн), $\pi_0 = 4000$ грн збільшує його на $10555{\cdot}2000$ грн, $\pi = \text{unif}(3000{,}100)$ – приблизно на $75824{\cdot}1500$ грн; вартість смуги s в імітацію не входить і змінює всі $T(k)$ пропорційно, не змінюючи k^*.

Середня черга за формулою (2.4) збігається з оцінкою за формулою Літтла $\overline{L} = \lambda_q \overline{W}_q$ (λ_q – інтенсивність виходу з черги, $\overline{W}_q$ – середній час у черзі) з розбіжністю до 0,021 %, зумовленою літаками, що залишилися в черзі наприкінці.

3.4.2 Порівняння з аналітичними моделями

Окремі випадки моделі є класичними моделями [1, 13]: за часу зайнятості exp(35) без відходу і дозаправки – M/M/m, з відходом за часом exp(140) – M/M/m+M (модель Ерланга A). Для M/M/m ($a = \lambda/\mu \approx 5{,}833$, $\rho = a/m$) використано формулу Ерланга C:

$$P_w = \frac{\frac{a^m}{m!}\cdot\frac{1}{1-\rho}}{\sum_{n=0}^{m-1}\frac{a^n}{n!}+\frac{a^m}{m!}\cdot\frac{1}{1-\rho}}, L_q = P_w\frac{\rho}{1-\rho} \qquad (3.1)$$

а для M/M/m+M – стаціонарні ймовірності процесу загибелі та розмноження ($\gamma = 1/140$):

$$P_n = P_0 \prod_{j=1}^{n} \frac{\lambda}{min(j,m)\mu + max(j-m,0)\gamma}, P_{ab} = \frac{\gamma L_q}{\lambda} \quad (3.2)$$

Для кожного варіанта виконано 10 реплікацій тривалістю 2 000 000 хв (M/M/m) і 1 000 000 хв (M/M/m+M) після перехідного періоду; відносне відхилення $\Delta = (y_{\text{sim}} - y_{\text{an}})/y_{\text{an}} \cdot 100\%$ наведено в таблицях 3.3 і 3.4.

Таблиця 3.3 – Порівняння результатів імітації з моделлю M/M/m (формула Ерланга C), 10 реплікацій по 2 000 000 хв

m	L_q **імітація, літаків**	L_q **формула, літаків**	**Δ(L_q), %**	P_w **імітація**	P_w **формула**	**ρ імітація / формула**	**95 % інтервал для L_q, ±**
6	31,421	32,349	−2,87	0,9228	0,9242	0,9717 / 0,9722	2,675
7	2,775	2,793	−0,64	0,5576	0,5586	0,8330 / 0,8333	0,057
8	0,8593	0,8595	−0,03	0,3188	0,3193	0,7289 / 0,7292	0,018
10	0,1233	0,1218	+1,19	0,0869	0,0870	0,5831 / 0,5833	0,0031

Таблиця 3.4 – Порівняння результатів імітації з моделлю M/M/m+M (модель Ерланга A), 10 реплікацій по 1 000 000 хв

m	L_q **імітація ± δ (β = 0,95), літаків**	L_q **формула, літаків**	**Δ(L_q), %**	P_{ab} **імітація**	P_{ab} **формула**	**Δ(P_{ab}), %**	**ρ імітація / формула**
2	15,323 ± 0,039	15,334	−0,07	0,6568	0,6572	−0,06	1,0000 / 1,0000
4	7,507 ± 0,042	7,535	−0,38	0,3214	0,3229	−0,48	0,9874 / 0,9874
6	2,255 ± 0,033	2,275	−0,85	0,0965	0,0975	−1,00	0,8768 / 0,8774
8	0,5015 ± 0,0090	0,5088	−1,43	0,0214	0,0218	−1,80	0,7122 / 0,7133

Завантаження і ймовірність очікування збігаються з аналітичними з відхиленням до 0,2 %, частка відходів – до 1,8 %. Відхилення середньої черги

більші (до −2,87 % за $m = 6$, $\rho = 0{,}972$, коли черга змінюється повільно), але аналітичне значення в усіх випадках лежить у 95-відсотковому довірчому інтервалі (останній стовпець таблиці 3.3 і стовпець «± δ» таблиці 3.4, $t_{0{,}975;9} = 2{,}262$).

3.4.3 Перевірка протоколу подій

Для ручної перевірки виконано прогін із детермінованими даними: $m = 2$, інтервал 6 хв, зайнятість 35 хв, $\theta = 70$ хв, $c = 1000$ грн, $\pi = 1500$ грн, $w_{\max} = 140$ хв, $T_{\mathrm{mod}} = 420$ хв. Фрагмент протоколу наведено в таблиці 3.5 (L – довжина черги, K – кількість зайнятих смуг, P – накопичений прибуток; повний протокол – у файлі `results/trace_deterministic_m2.txt` репозиторію).

Таблиця 3.5 – Фрагмент протоколу подій детермінованого прогону (m = 2, час моделювання 420 хв) з ручною перевіркою

t, хв	Зміна стану	Літак (a_i, хв)	L	K	N_r / N_d	P, грн	Ручний розрахунок
208,00	черга: дозаправка	23 (138)	22	2	15 / 0	4000	138 + 70 = 208; −1000
210,00	надходження → черга	35 (210)	23	2	15 / 0	4000	6 · 35 = 210
214,00	черга: дозаправка	24 (144)	23	2	16 / 0	3000	144 + 70 = 214
216,00	вихід з каналу D1 (літак 11); канал займає літак 13 з черги	13 (78)	22	2	16 / 0	4500	посадка о 181; 181 + 35 = 216; w = 216 − 78 = 138; +1500
216,00	надходження → черга	36 (216)	23	2	16 / 0	4500	6 · 36 = 216; смугу вже зайняв літак 13
228,00	надходження → черга	38 (228)	24	2	18 / 0	4000	6 · 38 = 228
230,00	черга: вихід до N_2 за часом очікування	15 (90)	23	2	18 / 1	4000	90 + 140 = 230; дозаправку о 160 не повернуто
232,00	черга: дозаправка	27 (162)	23	2	19 / 1	3000	162 + 70 = 232
236,00	черга: вихід до N_2 за часом очікування	16 (96)	23	2	19 / 2	3000	96 + 140 = 236
251,00	вихід з каналу D1 (літак 13); канал займає літак 19 з черги	19 (114)	22	2	22 / 4	1500	216 + 35 = 251; w = 137 < 140; літаки 17, 18 відійшли о 242, 248

Протокол відповідає правилам таблиці 2.2: літак виходить з каналу рівно через 35 хв після посадки, канал одразу займає перший літак черги, дозаправка настає через 70 хв очікування, вихід до N_2 – через 140 хв. У момент $t = 216$ хв вихід з каналу збігається з надходженням літака 36, і новий літак стає в чергу, а не займає смугу поза нею. У момент $t = 356$ хв вихід з каналу збігається з моментом виходу літака 36 з черги, і літак сідає з очікуванням рівно 140 хв, бо вихід з каналу обробляється першим. Підсумок прогону (70 надходжень, 2 + 22 посадки, 22 відходи, 24 літаки в черзі, 50 дозаправок, прибуток $2{\cdot}2000 + 22{\cdot}1500 - 50{\cdot}1000 = -13000$ грн) точно збігається з ручним розрахунком. Протокол прогону з випадковими даними (файл `results/trace_random_m2.txt`) також перевірено вручну.

3.4.4 Висновок щодо коректності алгоритму

Баланс літаків виконується точно, пропускна здатність і завантаження збігаються з теоретичними, кожна вхідна змінна і кожен параметр змінюють вихідні характеристики в очікуваному напрямі й на очікувану величину, окремі випадки відтворюють формули Ерланга C і A, протокол збігається з ручним розрахунком. Отже, алгоритм відповідає формалізованому опису, і модель придатна для експериментального дослідження. Статистичні оцінки вихідних характеристик моделі (середні, середньоквадратичні відхилення, довірчі інтервали, гістограми частот і перевірка закону розподілу) отримано за обґрунтованими в розділі 4 умовами експерименту і наведено в підрозділах 4.5 і 4.6 (таблиці 4.5–4.7, рисунок 4.5).

4 ЕКСПЕРИМЕНТАЛЬНЕ ДОСЛІДЖЕННЯ МОДЕЛІ

4.1 Мета використання моделі, фактор і відгук

Модель використовується для пошуку оптимальних умов [5]. Фактор – кількість додаткових смуг $k \in \{0, ..., 10\}$ ($k = 0$ – база для приросту прибутку), основний відгук – час окупності $T(k)$, допоміжні – річний прибуток $p(m)$ і показники обслуговування. Оптимум існує, бо витрати $S(k)$ зростають лінійно, а приріст прибутку обмежений (за досить великого m сідають усі літаки, граничний прибуток ≈ 175 млн грн/рік). Задача оптимізації – мінімізувати $T(k)$ за цілочисельного $k \in \{1, ..., 10\}$ (формула (2.8)). Кількість смуг набуває лише окремих цілих значень, тому за класифікацією факторів курсу це якісний фактор [7]: рівняння регресії і градієнтні методи, що спираються на неперервну зміну фактора, до нього не застосовуються, а вплив фактора оцінюється дисперсійним аналізом. Еволюційні методи доцільні, коли простір пошуку великий і перебрати його неможливо, а тут допустимих значень лише десять. Тому оптимум шукано покроковим пошуком: починаючи з $k = 1$, кількість смуг збільшують на одиницю, доки крок значуще зменшує $T(k)$ за парним t-критерієм (4.6), і зупиняються на першому k, після якого T значуще зростає. Траєкторія пошуку: 48,59 → 48,45 → 46,20 → 29,20 → 28,24 → 32,11 доби для $k = 1, ..., 6$ (t кроків: −27,2; −76,0; −396; −22,3; +962), тобто зупинка на $k = 5$. Локальний пошук сам не гарантує глобального оптимуму, тому обчислено також решту значень $k = 7, ..., 10$ (повний перебір): T далі лише зростає (таблиця 4.5), і мінімум $k = 5$ єдиний. Усі варіанти досліджено за єдиним планом (однакові умови, кількість прогонів і загальні випадкові числа), а вибір оптимуму підтверджено статистичними критеріями (підрозділ 4.6).

Варіанти порівнюються методом загальних випадкових чисел: реплікація l для всіх k запускається із зерном $1000 + l$, тож усі конфігурації обслуговують ту саму послідовність літаків. Вплив методу на розкид показано в таблиці 4.1 (останній стовпець – розкид за незалежних зерен, $\sqrt{\sigma^2(p(2+k)) + \sigma^2(p(2))}$).

Таблиця 4.1 – Вплив методу загальних випадкових чисел на розкид приросту прибутку (n = 100, T_{mod} = 528 600 хв)

k, шт.	**m, шт.**	**Коефіцієнт кореляції ρ(2) і ρ(2+k)**	**σ(ρ(2+k)), млн грн/рік**	**σ(Δp) із загальними числами, млн грн/рік**	**σ(Δp) для незалежних прогонів, млн грн/рік**
1	3	0,999	0,308	0,010	0,436
2	4	0,986	0,327	0,057	0,449
3	5	0,840	0,718	0,489	0,781
4	6	0,824	2,117	1,872	2,140
5	7	−0,697	0,323	0,581	0,446
6	8	−0,938	0,404	0,701	0,508
10	12	−0,992	0,592	0,898	0,667

Для перевантажених $k = 1{,}2$ розкид приросту зменшується у 8–44 рази. Для $k \geq 5$ кореляція з базою від'ємна, і $\sigma(\Delta p)$ дещо більша, але для вибору оптимуму важливі сусідні варіанти: кореляція $T_l(5)$ і $T_l(6)$ дорівнює 0,948, а розкид різниці $T_l(6) - T_l(5)$ – $1{,}10 \cdot 10^{-4}$ року проти $3{,}80 \cdot 10^{-4}$ за незалежних прогонів.

4.2 Тривалість перехідного періоду

Тривалість перехідного періоду визначено методом Велча [4, 14]: довжину черги $L(t)$ фіксували кожні 50 хв, усереднювали за прогонами і згладжували ковзним середнім з півшириною вікна $w = 10$ точок (вікно $2w + 1 = 21$ точка, 1050 хв). Кінець перехідного періоду – перший момент, коли крива входить у смугу «сталий рівень ± max(5 %, 2σ)», де σ – розкид усередненої кривої в сталому режимі (рис. 4.1, табл. 4.2).

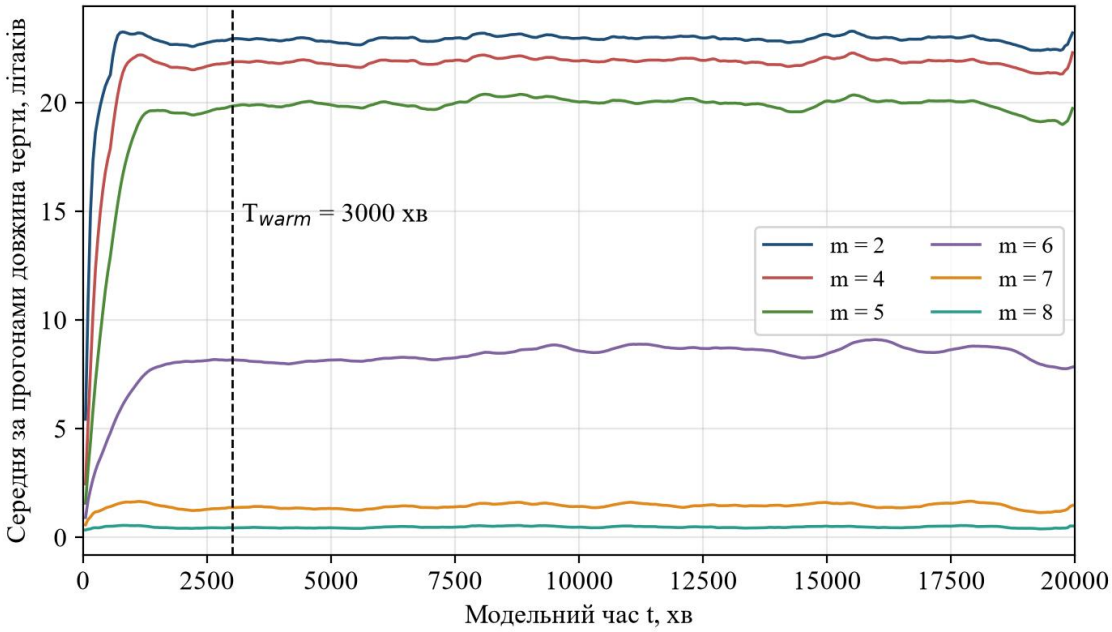


Рисунок 4.1 – Середня за прогонами довжина черги, згладжена ковзним середнім Велча

Таблиця 4.2 – Тривалість перехідного періоду за методом Велча

Кількість смуг m, шт.	**Навантаження $\rho = \lambda t_D/m$**	**Кількість прогонів × тривалість, хв**	**Сталий рівень черги, літаків**	**Ширина смуги, літаків**	**Тривалість перехідного періоду, хв**
2	2,917	200 × 20 000	22,89	1,14	600
4	1,458	200 × 20 000	21,84	1,09	750
5	1,167	200 × 20 000	19,88	0,99	1100
6	0,972	400 × 60 000	8,27	0,71	1300
7	0,833	200 × 20 000	1,44	0,43	200
8	0,729	200 × 20 000	0,46	0,19	50

У перевантажених конфігураціях черга росте до рівня $\approx \lambda w_{\max} \approx 23$ літаки, найінерційніша конфігурація – $m = 6$ ($\rho \approx 0{,}97$). Помилка в більший бік допустима [5], тому прийнято $T_{\text{warm}} = 3000$ хв – більш ніж удвічі більше за найбільше значення 1300 хв.

4.3 Тривалість прогону

Тривалість прогону визначено за умовою збіжності накопичених середніх у кількох незалежних прогонах [5]:

$$T: \forall t \geq T, \forall i, j: \left|\overline{p}_i(t) - \overline{p}_j(t)\right| < \varepsilon, \tag{4.1}$$

де $\overline{p}_i(t) = 525600 \cdot P_i(t)/t$ – накопичене середнє річного прибутку в i-му прогоні. Виконано по 5 прогонів тривалістю рік для $m = 2,4,6,7,8$ (рис. 4.2) і 5 прогонів тривалістю 5 років для $m = 6$ (табл. 4.3).

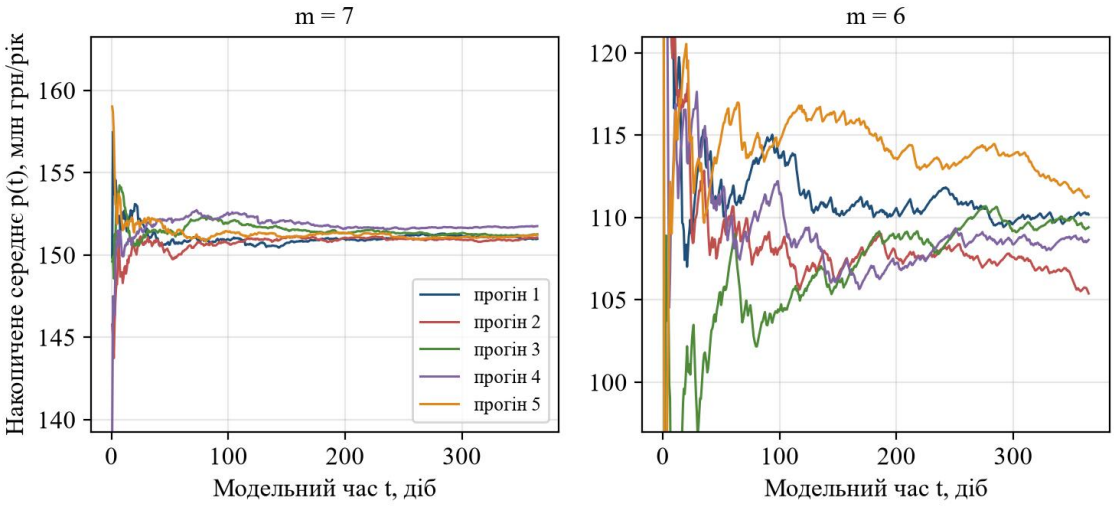


Рисунок 4.2 – Накопичене середнє річного прибутку в п’яти незалежних прогонах

Таблиця 4.3 – Тривалість прогону за умовою (4.1) для п’яти прогонів

m, шт. (тривалість)	**Середнє p наприкінці, млн грн/рік**	**T для ε = 5 млн грн/рік, хв**	**T для ε = 2 млн грн/рік, хв**	**T для ε = 1 млн грн/рік, хв**	**Розкид наприкінці, млн грн/рік**
2 (1 рік)	−42,33	20 000	136 000	309 000	0,62
4 (1 рік)	2,89	21 000	136 000	311 000	0,67
6 (1 рік)	108,97	–	–	–	5,90
6 (5 років)	107,66	735 000	–	–	2,85
7 (1 рік)	151,25	15 000	106 000	268 000	0,78
8 (1 рік)	161,92	20 000	213 000	–	1,10

Для $m = 2,4,7$ умова з $\varepsilon = 1$ млн грн/рік виконується за 268 000–311 000 хв, тобто приблизно на половині року, для $m = 8$ – умова з $\varepsilon = 2$ млн грн/рік. Конфігурація $m = 6$ ($\rho \approx 0,97$) працює поблизу межі перевантаження, і розкид прибутку між прогонами дуже великий: навіть через 5 років він становить 2,85 млн грн/рік. Тому прийнято $T_{\text{ст}} = 525600$ хв, $T_{\text{mod}} = 528600$ хв, а точність усіх оцінок забезпечено кількістю прогонів з перевіркою довірчих інтервалів (підрозділ 4.4). Для $m = 6$ і $m = 8$ умову (4.1) з $\varepsilon_p = 1$ млн грн/рік за рік не досягнуто, тому її замінено альтернативною вимогою до точності середнього за

n незалежними прогонами (різні зерна): напівширина довірчого інтервалу (формула (4.3)) має не перевищувати ε_p. Рік збору статистики залишено, бо саме за рік визначено прибуток p в умові завдання, а збільшення тривалості прогону для $m = 6$ навіть до 5 років не забезпечує збіжності. За серію з $n = 100$ прогонів для кожного k обробляється понад 8,7 млн літаків.

4.4 Кількість прогонів

Кількість прогонів визначено за формулою, що випливає з нерівності Чебишева [5]:

$$n = \frac{\sigma^2}{\varepsilon^2(1-\beta)} = 20\left(\frac{\sigma}{\varepsilon}\right)^2, \tag{4.2}$$

де σ оцінено за пілотною серією з $n_0 = 20$ реплікацій (табл. 4.4), $\beta = 0{,}95$.

Таблиця 4.4 – Оцінки розкиду за пілотною серією і потрібна кількість прогонів ($n_0 = 20$, $T_{mod} = 528\,600$ хв)

k, шт.	m, шт.	σ(p), млн грн/рік	n для ε_p = 1 млн грн/рік	T(k), років	σ(T), 10^{-4} років	n для ε_T = 0,0005 року
0	2	0,288	1,7	–	–	–
1	3	0,289	1,7	0,13312	0,50	0,2
2	4	0,314	2,0	0,13274	1,55	1,9
3	5	0,780	12,2	0,12654	10,43	87,1
4	6	2,224	98,9	0,07978	10,71	91,7
5	7	0,274	1,5	0,07738	2,03	3,3
6	8	0,337	2,3	0,08798	2,61	5,4
7	9	0,398	3,2	0,09967	3,22	8,3
8	10	0,460	4,2	0,11207	3,90	12,2
9	11	0,511	5,2	0,12501	4,62	17,0
10	12	0,531	5,6	0,13832	5,21	21,7

Найбільшу кількість прогонів потребують конфігурації на межі перевантаження ($m = 5{,}6$): $n \approx 92$ для T і $n \approx 99$ для p, тому взято $n = 100$. Напівширини довірчих інтервалів обчислено за розподілом Стьюдента ($t_{0{,}975;99} = 1{,}984$):

$$\delta = t_{(1+\beta)/2;n-1} \cdot \frac{s}{\sqrt{n}} \tag{4.3}$$

Отримані значення (табл. 4.5) не перевищують 0,42 млн грн/рік і 0,0002 року, тобто вимоги до точності виконано для всіх k.

4.5 Серія експериментів повного перебору

Основна серія – $k = 0, ..., 10$ по $n = 100$ реплікацій (зерна 1001–1100, $T_{\text{warm}} = 3000$ хв, $T_{\text{mod}} = 528600$ хв), 1100 прогонів приблизно за 11 с. Середнє, довірчий інтервал і середньоквадратичне відхилення відгуку наведено в таблиці 4.5, залежність $T(k)$ – на рисунку 4.3, показники обслуговування – на рисунку 4.4 і в таблиці 4.6.

Таблиця 4.5 – Результати повного перебору кількості додаткових смуг (n = 100, T_{mod} = 528 600 хв, T_{warm} = 3000 хв)

k, шт.	**m, шт.**	**p(m) ± δ, млн грн/рік**	**Δp(k), млн грн/рік**	**T(k) ± δ, років**	**s(T), 10^{-4} років**	**T(k), діб**
0	2	−42,515 ± 0,061	–	–	–	–
1	3	−19,979 ± 0,061	22,535	0,13312 ± 0,00001	0,58	48,59
2	4	2,684 ± 0,065	45,199	0,13275 ± 0,00003	1,66	48,45
3	5	28,593 ± 0,143	71,108	0,12657 ± 0,00017	8,70	46,20
4	6	107,524 ± 0,420	150,038	0,07999 ± 0,00020	10,01	29,20
5	7	151,333 ± 0,064	193,847	0,07738 ± 0,00005	2,32	28,24
6	8	162,099 ± 0,080	204,614	0,08797 ± 0,00006	3,02	32,11
7	9	168,241 ± 0,095	210,756	0,09964 ± 0,00007	3,69	36,37
8	10	171,673 ± 0,106	214,187	0,11205 ± 0,00009	4,38	40,90
9	11	173,488 ± 0,114	216,003	0,12500 ± 0,00010	5,08	45,63
10	12	174,403 ± 0,117	216,918	0,13830 ± 0,00011	5,72	50,48

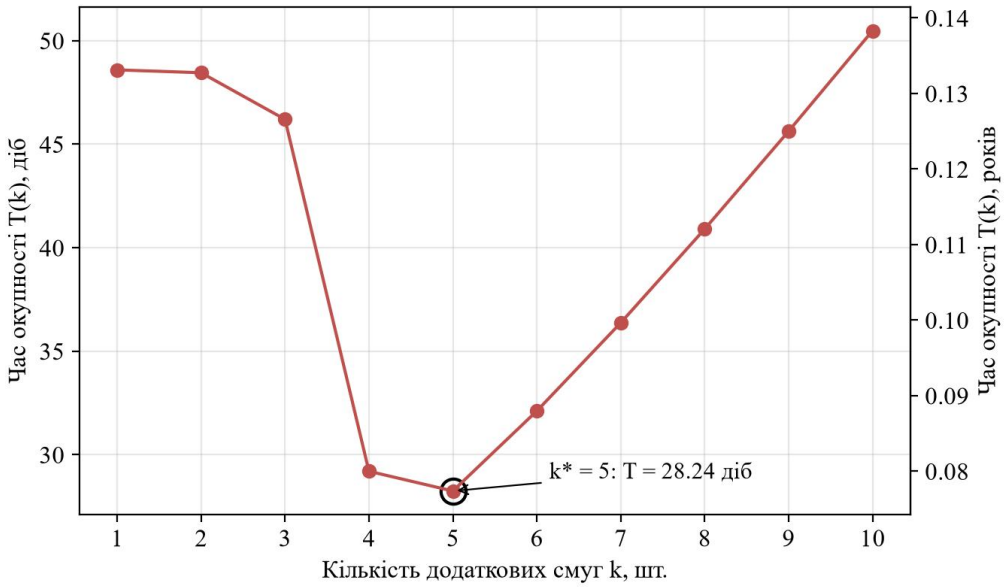


Рисунок 4.3 – Залежність часу окупності додаткових смуг від їх кількості (n = 100, T_{mod} = 528 600 хв)

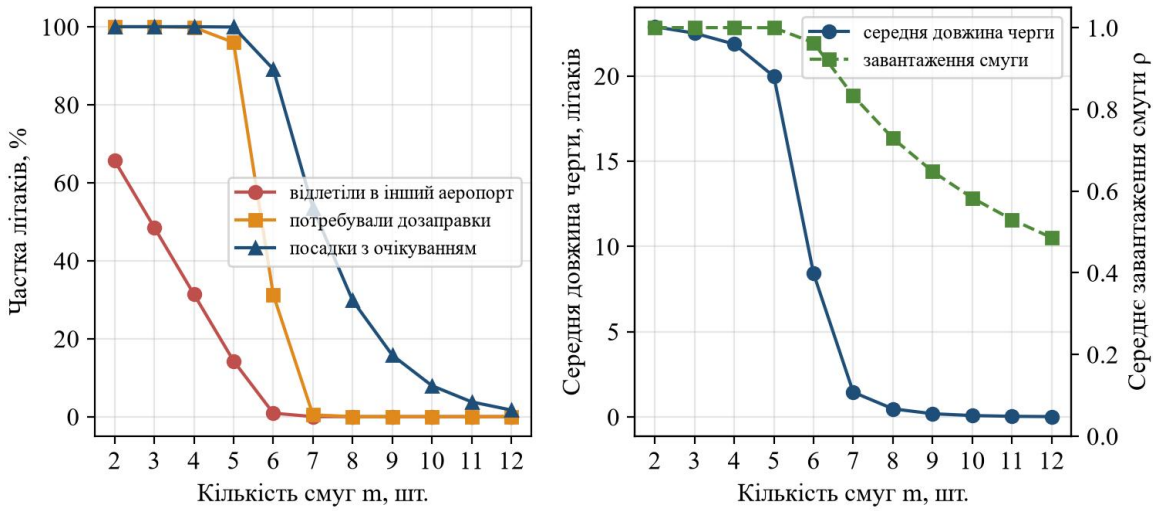


Рисунок 4.4 – Показники обслуговування літаків залежно від кількості смуг (n = 100, T_{mod} = 528 600 хв)

Таблиця 4.6 – Показники обслуговування та витрати на дозаправку (середні за n = 100 прогонами, T_{mod} = 528 600 хв)

m, шт.	$\overline{L}$, **літаків**	L_{max} **за всі прогони, літаків**	$\overline{\rho}$	$\overline{w}$, **хв**	$\overline{w}_w$, **хв**	**Частка відходів, %**	**Частка дозаправок, %**	**Витрати на дозаправку, млн грн/рік**
2	22,88	53	1,000	137,34	132,25	65,70	99,998	87,567
3	22,50	52	1,000	135,07	130,41	48,55	99,986	87,556
4	21,86	52	1,000	131,22	127,20	31,41	99,828	87,419
5	19,97	53	1,000	119,86	116,60	14,28	95,967	84,040
6	8,41	46	0,963	50,46	55,72	0,889	31,271	27,390

Продовження таблиці 4.6

m, шт.	$\overline{L}$, **літаків**	$L_{\max}$ **за всі прогони, літаків**	$\overline{\rho}$	$\overline{w}$, **хв**	$\overline{w}_w$, **хв**	**Частка відходів, %**	**Частка дозаправок, %**	**Витрати на дозаправку, млн грн/рік**	
7	1,45	35	0,833	8,67	16,22	0,0004	0,452	0,397	
8	0,47	22	0,729	2,81	9,42	0	0,003	0,002	
9	0,18	19	0,648	1,09	6,93	0	0	0	
10	0,07	16	0,583	0,45	5,62	0	0	0	
11	0,03	13	0,530	0,18	4,78	0	0	0	
12	0,01	12	0,486	0,07	4,20	0	0	0	

При $k = 1, ..., 3$ аеропорт перевантажений: кожна смуга додає ≈ 15 017 посадок на рік, але ці літаки однаково встигають дозаправитися, тож прибуток зростає лише на 22,5–25,9 млн грн/рік, і $T \approx 0{,}133$ року ≈ 48 діб. При $k = 4$ ($\rho = 0{,}972$) відходи падають до 0,89 %, дозаправки – до 31 %, і $T(4) = 29{,}20$ доби, але черга ще велика (8,4 літака). Мінімум досягається при $k^* = 5$ ($\rho = 0{,}833$): $T(5) = 0{,}07738 \pm 0{,}00005$ року ≈ 28,24 доби, дозаправки (0,45 %) і відходи майже зникають, $\overline{w} = 8{,}7$ хв. Далі кожна смуга додає дедалі менше (шоста додаткова, восьма загалом, – 10,77 млн грн/рік), а витрати зростають лінійно, тому при $k = 10$ час окупності сягає 50,48 доби.

4.6 Статистична обробка результатів

Закон розподілу відгуку. Для оптимального варіанта побудовано гістограми $T_l(5)$ і $p_l(7)$ за 100 реплікаціями (8 інтервалів за формулою Стерджеса, рис. 4.5) і перевірено гіпотезу про нормальний закон критерієм χ^2 Пірсона за формулою:

$$\chi^2 = \sum_{i=1}^{r} \frac{(n_i - np_i)^2}{np_i} < \chi^2_{\text{кр}}(\alpha; r - 1 - 2), \tag{4.4}$$

де n_i і np_i – спостережена й очікувана частоти i-го інтервалу, r – кількість інтервалів після об’єднання тих, де $np_i < 5$ (табл. 4.7).

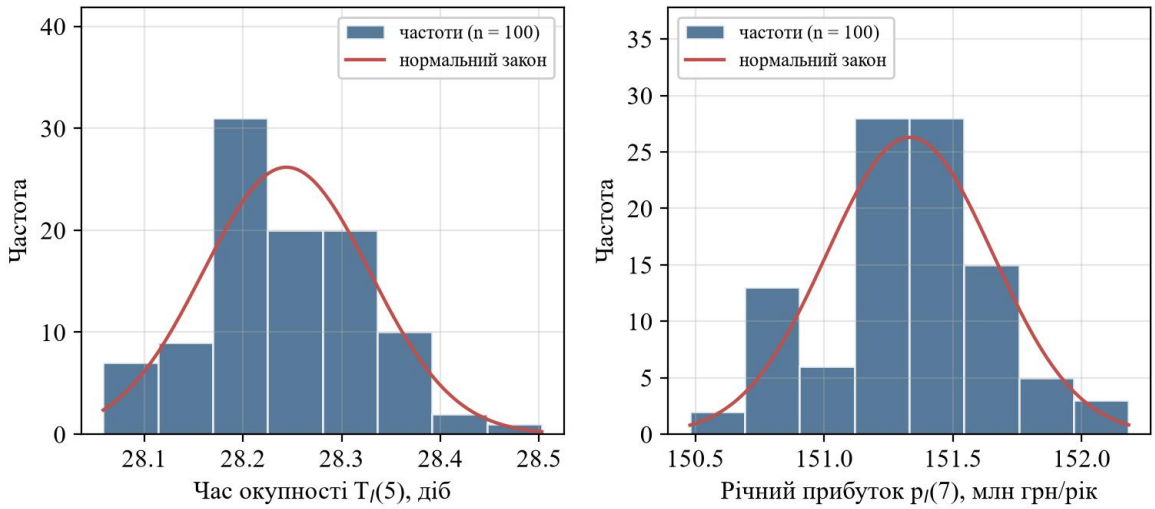


Рисунок 4.5 – Гістограми частот часу окупності $T_l(5)$ і річного прибутку $p_l(7)$ та щільність нормального закону

Таблиця 4.7 – Статистичні оцінки відгуку оптимального варіанта k* = 5 (n = 100, T_{mod} = 528 600 хв)

Характеристика	**Час окупності $T_l(5)$, діб**	**Річний прибуток $p_l(7)$, млн грн/рік**
Середнє ± δ (β = 0,95)	28,244 ± 0,017	151,333 ± 0,064
Середньоквадратичне відхилення s	0,085	0,323
Мінімум / максимум	28,059 / 28,503	150,479 / 152,183
Коефіцієнт варіації, %	0,30	0,21
Асиметрія / ексцес	0,21 / 0,02	−0,12 / 0,01
Частоти 8 інтервалів	7, 9, 31, 20, 20, 10, 2, 1	2, 13, 6, 28, 28, 15, 5, 3
χ^2 (r = 6, 3 ступені вільності)	5,95	11,12
Критичне значення χ^2 для α = 0,05	7,81	7,81
Висновок (α = 0,05)	нормальний закон не відкидається	нормальний закон відкидається

Для часу окупності гіпотеза про нормальний закон не відкидається ($\chi^2 = 5{,}95 < 7{,}81$). Для прибутку гіпотеза про нормальний закон відкидається ($\chi^2 = 11{,}12 > 7{,}81$), тож закон розподілу $p_l(7)$ за критерієм χ^2 не встановлено. Відхилення зосереджене в одному інтервалі (6 спостережень замість ≈ 16) і може бути наслідком групування 100 значень у 8 інтервалів: критерій Шапіро – Уїлка, що не потребує групування, нормальності не заперечує (рівень значущості $p_{зн} = 0{,}80$). Для висновків роботи закон розподілу $p_l(7)$ не є

визначальним, бо основним відгуком є час окупності, для якого нормальний закон не відкидається; довірчі інтервали побудовано для середнього, яке за $n = 100$ розподілене практично нормально.

Вплив фактора. Однорідність дисперсій перевірено критерієм Кохрена $G = max_j s_j^2 / \sum_j s_j^2 < G_{\text{кр}}$. Для всіх рівнів $k = 1, ..., 10$ отримано $G = 0{,}352 > G_{\text{кр}}(\alpha = 0{,}05; 99; 10) = 0{,}138$ – експеримент невідтворюваний через різні режими роботи (коефіцієнт варіації T – 0,04–0,13 % у перевантаженому режимі, 0,7–1,3 % на межі перевантаження, 0,30–0,41 % у сталому). Тому, як рекомендує методика, інтервал варіювання звужено до області сталого режиму $k = 5, ..., 10$, що містить оптимум. Там розкид T пропорційний середньому, тож для стабілізації дисперсії відгук прологарифмовано ($y = \ln T$, дисперсія якого приблизно дорівнює квадрату коефіцієнта варіації) [15]; отримано $G = 0{,}205 < G_{\text{кр}}(\alpha = 0{,}05; 99; 6) = 0{,}222$.

Значення різних рівнів в одній реплікації залежні (загальні випадкові числа), тому застосовано двофакторний дисперсійний аналіз без повторень, де другий фактор – реплікація (блок) [15]:

$$F = \frac{MS_k}{MS_e} > F_{\text{кр}}(\alpha; q-1; (q-1)(n-1)), \tag{4.5}$$

де MS_k і MS_e – середні квадрати фактора і залишку після вилучення впливу реплікацій, q – кількість рівнів. Отримано $F = 9{,}3 \cdot 10^6 > F_{\text{кр}}(0{,}05; 5; 495) = 2{,}23$; вплив реплікацій теж значущий ($F = 158$), що підтверджує залежність вибірок. Парний t-критерій – окрема процедура порівняння двох залежних вибірок, а не етап регресійного аналізу, тому невідтворюваність на всьому діапазоні k (критерій Кохрена) його застосуванню не заважає. Сам вплив кількості смуг очевидний, тому головне для оптимізації – перевірка оптимуму парним t-критерієм, який використовує лише різниці значень однієї реплікації й однорідності дисперсій не потребує:

$$t = \frac{\overline{d}}{s_d/\sqrt{n}}, d_l = T_l(k) - T_l(k^*) \tag{4.6}$$

Різниця значуща при $|t| > t_{0{,}975;99} = 1{,}984$ (табл. 4.8).

Таблиця 4.8 – Парні порівняння часу окупності з оптимальним варіантом k* = 5 (n = 100, T_{mod} = 528 600 хв)

k, шт.	**Середня різниця $\bar{d}$ = T(k) − T(5), років**	**$\bar{d}$, діб**	**s_d, 10^{-4} років**	**t**	**Частка реплікацій, де T(k) > T(5), %**
1	0,05574	20,35	2,47	2258	100
2	0,05537	20,21	3,29	1682	100
3	0,04919	17,96	10,38	474	100
4	0,00261	0,95	11,71	22,3	99
6	0,01059	3,87	1,10	962	100
7	0,02226	8,13	1,73	1285	100
8	0,03467	12,66	2,38	1456	100
9	0,04762	17,38	3,07	1553	100
10	0,06092	22,24	3,70	1647	100

Усі різниці додатні й значущі; великі t – наслідок малого розкиду різниць за загальних випадкових чисел. Найближчий конкурент $k = 4$ гірший на 0,95 доби ($t = 22{,}3$), і $T(4) > T(5)$ у 99 зі 100 реплікацій. Отже, з довірчою ймовірністю 0,95 оптимальною є побудова $k^* = 5$ додаткових смуг.

Вплив кількісних факторів на оптимальний варіант. Інтенсивність потоку λ і час зайнятості смуги t_D – кількісні фактори, значення яких для реального аеропорту відомі неточно. Їхній вплив на час окупності оптимального варіанта оцінено повним факторним експериментом 2^2 (табл. 4.9): x_1 – інтенсивність потоку λ (9,8 і 10,2 літака/год, ±2 %), x_2 – час зайнятості смуги t_D (34,3 і 35,7 хв, ±2 %), відгук $y = \ln T(5)$, по $n = 100$ прогонів у кожній точці плану (зерна 1001–1100; приріст прибутку відраховано від двох смуг за тих самих λ і t_D). Інтервали варіювання вибрано вузькими, бо на ширшому плані дисперсії неоднорідні (див. нижче). На вибраному плані дисперсії однорідні: $G = 0{,}286 < G_{\text{кр}}(0{,}05; 99; 4) = 0{,}322$.

Таблиця 4.9 – План і результати повного факторного експерименту 2^2 для k = 5 (n = 100, T_{mod} = 528 600 хв)

№	λ, літаків/год	t_D, хв	x_1	x_2	$\overline{y}$	T(5), діб
1	9,8	34,3	−1	−1	−2,5470	28,59
2	9,8	35,7	−1	+1	−2,5387	28,82
3	10,2	34,3	+1	−1	−2,5791	27,68
4	10,2	35,7	+1	+1	−2,5649	28,08

Коефіцієнти рівняння регресії обчислено за формулою $b_j = \frac{1}{N}\sum_{i=1}^{N} x_{ij}\,\overline{y}_i$ ($N = 4$):

$$\hat{y} = -2{,}5574 - 0{,}0146x_1 + 0{,}0056x_2 + 0{,}0015x_1x_2 \qquad (4.7)$$

Дисперсія коефіцієнтів $s_b^2 = s^2_{\text{відтв}}/(Nn)$, $s_b = 1{,}51\cdot10^{-4}$; t_j = 16897; 96,3; 37,0; 9,73 > $t_{\text{кр}}(0{,}05; 396) = 1{,}966$, тож усі коефіцієнти значущі. Рівняння насичене (чотири коефіцієнти на чотири точки плану), тому ступенів вільності для перевірки адекватності за критерієм Фішера немає і жоден доданок не відкидається. У межах плану ($x_1, x_2 \in [-1; 1]$) перехід потоку з нижнього рівня на верхній (з 9,8 до 10,2 літака/год, на 4 %) скорочує час окупності приблизно на 2,9 % ($e^{2b_1} - 1$), а перехід часу зайнятості смуги з 34,3 до 35,7 хв (на 4 %) подовжує його на 1,1 % ($e^{2b_2} - 1$): потік впливає на окупність у 2,6 раза сильніше. Додатний коефіцієнт взаємодії b_{12} означає, що за більшого t_D вплив потоку слабшає.

Взаємодія потоку і кількості смуг. Серії повного перебору для потоку 8 і 12 літаків/год (підрозділ 5.3) разом з основною серією утворюють повний факторний план 3 × 5 з факторами $\lambda \in \{8{,}10{,}12\}$ літаків/год і $k = 3, \ldots, 7$. Класичний двофакторний дисперсійний аналіз з оцінкою взаємодії $\lambda \times k$ до нього незастосовний: дисперсії неоднорідні навіть для $\ln T$ ($G = 0{,}376 > G_{\text{кр}}(0{,}05; 99; 15) = 0{,}094$), бо план охоплює і перевантажені, і недовантажені конфігурації.

Тому взаємодію потоку і кількості смуг оцінено парними порівняннями в межах кожного рівня λ, які однорідності дисперсій не потребують. Додаткові серії для $\lambda = 9{,}8$ і $10{,}2$ літака/год ($k = 0, ..., 6$) показали, що оптимум чутливий навіть до малої зміни потоку: за $\lambda = 9{,}8$ літака/год найменший час окупності має $k = 4$ (27,90 доби проти 28,69 для $k = 5$; різниця значуща, $t = 24{,}2$), а за $\lambda = 10{,}2$ – знову $k = 5$ (27,85 доби проти 31,07 для $k = 4$). Варіанти $k = 4$ і $k = 5$ близькі за окупністю, і вибір між ними визначається точністю прогнозу потоку.

4.7 Програмне забезпечення для проведення експериментів

Експерименти виконує програма AirportSim: `transient` – дані для методу Велча і умови (4.1), `series` – повний перебір k (метод `Experiments.Series` виконує прогони паралельно через PLINQ зі збереженням порядку, тож результати не залежать від кількості ядер), `verify` – верифікацію. Основну серію запускає команда `AirportSim series --tag main --reps 100 --seedbase 1000 --twarm 3000 --tstat 525600`. Статистичну обробку, критерії і рисунки виконує скрипт `analysis/analyze.py` [8], числа зберігаються в `results/summary.json`; лістинги – у додатку Б.

5 ІНТЕРПРЕТАЦІЯ РЕЗУЛЬТАТІВ МОДЕЛЮВАННЯ, ФОРМУЛЮВАННЯ ВИСНОВКІВ ТА ПРОПОЗИЦІЙ

5.1 Інтерпретація результатів і пропозиція 1

Дві смуги приймають 3,4 літака за годину із 10, тому черга постійно тримається на рівні ≈ 23 літаків, майже кожен літак дозаправляється, а 65,7 % після дозаправки все одно відходять. Витрати на дозаправки (87,6 млн грн/рік) майже вдвічі перевищують дохід від посадок (45,1 млн грн/рік), і аеропорт втрачає 42,5 млн грн на рік. Вузьке місце – кількість смуг, а нові смуги не лише дають дохід, а й усувають збиткові дозаправки, тому окупність дуже швидка.

Пропонується побудувати $k^* = 5$ додаткових смуг (7 загалом, табл. 5.1). Варіант із чотирма додатковими смугами (6 загалом) окуповується лише на 0,95 доби довше, але різниця значуща (табл. 4.8), а обслуговування в ньому значно гірше: черга 8,4 літака, 31 % дозаправок, $\overline{w} = 50{,}5$ хв.

Таблиця 5.1 – Порівняння наявного аеропорту і запропонованого варіанта (n = 100, T_{mod} = 528 600 хв)

Показник	**Наявний аеропорт, m = 2**	**Пропозиція, m = 7**
Інвестиції в нові смуги S, млн грн	0	15,0
Річний прибуток p, млн грн/рік	−42,515	151,333
Час окупності T, років (діб)	–	0,07738 (28,24)
Кількість посадок за рік, літаків	30 034	87 570
Частка відходів, %	65,70	0,0004
Частка дозаправок, %	99,998	0,45
Середнє очікування $\overline{w}$, хв	137,34	8,67

5.2 Пропозиція 2: скоротити час зайнятості смуги

Швидкісні руліжні доріжки дають змогу звільняти смугу швидше. Модифіковану модель ($t_D = 30$ хв) досліджено серією з тими самими зернами 1001–1100, тож вибірки початкової і модифікованої моделей попарно залежні, і їх порівняно парним t-критерієм (4.6). Шість смуг із $t_D = 30$ хв дають 149,99 ± 0,06 млн грн/рік, а сім смуг із $t_D = 35$ хв – 151,33 ± 0,06 млн грн/рік:

модифікація з однією смугою менше програє лише 1,34 млн грн/рік (0,9 %), різниця значуща ($t = -308$). Дисперсійний аналіз з якісним фактором «модель» (початкова / модифікована) і реплікацією як блоком дає той самий висновок: $F = 9{,}5 \cdot 10^4 > F_{\text{кр}}(0{,}05; 1; 99) = 3{,}94$ за однорідних дисперсій ($G = 0{,}500 < 0{,}598$); для двох рівнів $F = t^2$. Шість смуг – найкраща кількість для $t_D = 30$ хв: за приросту від наявного аеропорту і нульових витрат на реконструкцію час окупності становить 25,04 доби для 5 смуг, 22,75 – для 6 і 26,79 – для 7. Навантаження в обох варіантах однакове ($\rho = 0{,}833$), тож скорочення зайнятості на 5 хв майже заміняє одну смугу.

Для порівняння окупності варіантів треба відраховувати приріст прибутку від того самого наявного аеропорту (2 смуги, 35 хв) і враховувати вартість реконструкції X: для варіанта «6 смуг, 30 хв» $\Delta p = 149{,}99 - (-42{,}52) = 192{,}50$ млн грн/рік, і він окупається швидше за пропозицію 1, якщо $(12 + X)/192{,}50 < 0{,}07738$, тобто $X < 2{,}90$ млн грн. Вартості реконструкції в умові немає, тому рішення потребує кошторису; модель дає змогу швидко переоцінити варіант (параметр `--occupancy`).

5.3 Чутливість рекомендації до потоку і тлумачення прибутку

Для потоку 8 і 12 літаків/год виконано серії повного перебору з тими самими умовами ($n = 100$, зерна 1001–1100; напівширини довірчих інтервалів не перевищують 0,11 доби, табл. 5.2). Вплив малих (±2 %) змін потоку на окупність оптимального варіанта оцінено повним факторним експериментом 2^2 (підрозділ 4.6).

За потоку 9,8 літака/год оптимум зміщується вже до $k = 4$, а за 10,2 – залишається $k = 5$ (підрозділ 4.6).

Таблиця 5.2 – Час окупності T(k), діб, за різної інтенсивності потоку (n = 100, T_{mod} = 528 600 хв)

Інтенсивність потоку	**k = 3**	**k = 4**	**k = 5**	**k = 6**	**k = 7**	**k***	**Втрата для k = 5, діб**
8 літаків/год	26,53	29,36	34,88	40,77	46,94	3	8,36
10 літаків/год (умова)	46,20	29,20	28,24	32,11	36,37	5	0
12 літаків/год	48,54	47,34	32,72	27,72	30,51	6	5,00

Оптимум зміщується до $k^* = 3$ і $k^* = 6$ (переваги над $k = 5$ значущі: $t = 299$ і $t = 90$), і в усіх випадках це найменше m з навантаженням $\rho = \lambda t_D/m$ = 0,83–0,93. Отже, оптимальна рекомендація нестійка до зміни потоку: оптимум зсувається вже при зменшенні потоку на 2 % (до $k = 4$), тоді як при збільшенні на 2 % залишається $k = 5$. Проте $k = 5$ дає найменшу максимальну втрату окупності серед $k = 3, ..., 7$ за всіх досліджених значень потоку (8,4 доби проти 14,2 для $k = 6$ і 19,6 для $k = 4$), тому за невизначеного трафіку доцільно побудувати п’ять додаткових смуг і зарезервувати майданчик ще під одну (шосту додаткову, восьму загалом): за 12 літаків/год вона окупиться приблизно за 16 діб.

Тлумачення прибутку на рекомендацію не впливає: з повним річним прибутком $p(2 + k)$ замість приросту час окупності дорівнює 2,2697 року при $k = 2$, 0,1116 при $k = 4$, 0,0991 (36,18 доби) при $k = 5$ і 0,1110 при $k = 6$, тобто мінімум знову при $k^* = 5$.

ВИСНОВКИ

У роботі побудовано і досліджено імітаційну модель посадки літаків в аеропорті на основі мережі масового обслуговування з метою визначити кількість смуг, за якої час окупності додаткових смуг мінімальний.

Формалізована модель складається з надходження, універсальної багатоканальної СМО з $m = 2 + k$ пристроями і черги та двох виходів. Стандартних елементів мережі МО виявилося недостатньо: прибуток задано типом запиту, а дозаправку і відхід – модифікованим елементом «черга з обмеженим часом перебування» з двома власними подіями в моменти $a_i + \theta_i$ і $a_i + 140$, тож модель має чотири події замість двох стандартних. Найскладнішим у розробці алгоритму було саме це доповнення і порядок одночасних змін стану: якби черга оброблялася раніше за вихід з каналу, літак, для якого звільнилася смуга, міг би «відійти» в той самий момент. Завдяки об’єктно-орієнтованому підходу специфіку задачі вдалося додати класом-нащадком черги, не змінюючи універсального алгоритму; стохастична мережа Петрі чи Петрі-об’єктна модель [16] описали б ці зміни стану стандартними переходами, але ціною більшої кількості елементів. Коректність алгоритму підтверджено верифікацією за кожною вхідною змінною і параметром, ручною перевіркою протоколу подій і порівнянням з формулами Ерланга С і А.

Умови експерименту (перехідний період, тривалість і кількість прогонів) обґрунтовано методами тактичного планування, тож оцінки відгуку мають потрібну точність. Оптимальною є побудова $k^* = 5$ додаткових смуг (7 загалом) з часом окупності близько 28 діб. Наявний аеропорт має збиток 42,5 млн грн/рік, а з 7 смугами – прибуток 151,33 млн грн/рік, відходи практично зникають, середнє очікування скорочується з 137 до 9 хв. Перевагу $k = 5$ над найближчим $k = 4$ підтверджено парним t-критерієм. Повний факторний експеримент 2^2 за кількісними факторами показав, що в околі умов задачі інтенсивність потоку впливає на час окупності оптимального варіанта в 2,6 раза сильніше за час зайнятості смуги.

Рекомендації: побудувати п’ять додаткових смуг і зарезервувати майданчик ще під одну, бо оптимум чутливий до інтенсивності потоку (3 і 6 смуг за 8 і 12 літаків/год, 4 смуги вже за 9,8 літака/год), а $k = 5$ дає найменшу максимальну втрату; як альтернативу оцінити скорочення зайнятості смуги до 30 хв, яке з однією смугою менше дає прибуток лише на 0,9 % менший і вигідніше, якщо реконструкція коштує менше 2,9 млн грн.

Імітаційне моделювання дало змогу врахувати детерміновані затримки, випадковий поріг дозаправки, відхід за часом очікування і залежність прибутку від типу запиту, які важко описати аналітично; модель швидка (рік імітується за 0,1–0,4 с) і придатна для оцінювання інших рішень. Результати обмежені припущеннями про стаціонарний потік, детерміновану зайнятість смуги і вартість смуги без експлуатаційних витрат.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Fundamentals of Queueing Theory / D. Gross, J. F. Shortle, J. M. Thompson, C. M. Harris. 4th ed. Hoboken : John Wiley & Sons, 2008. 528 p.

2. Стеценко І. В. Моделювання систем : навч. посіб. Черкаси : ЧДТУ, 2010. 399 с.

3. Томашевський В. М. Моделювання систем. Київ : Видавнича група BHV, 2005. 352 с.

4. Law A. M. Simulation Modeling and Analysis. 5th ed. New York : McGraw-Hill Education, 2015. 804 p.

5. Стеценко І. В., Дифучина О. Ю., Дифучин А. Ю. Моделювання систем. Курсова робота : рек. до виконання курсов. роботи : навч. посіб. для здобувачів ступеня бакалавра за освіт. програмою «Інженерія програмного забезпечення інформаційних систем» спец. 121 Інженерія програмного забезпечення ; КПІ ім. Ігоря Сікорського. Київ : КПІ ім. Ігоря Сікорського, 2024. 109 с. URL: https://ela.kpi.ua/items/907b5d1f-ca48-4165-a67e-e790ac6fbec5 (дата звернення: 06.11.2026).

6. C# documentation. Microsoft Learn. URL: https://learn.microsoft.com/en-us/dotnet/csharp/ (дата звернення: 06.11.2026).

7. Стеценко І. В., Дифучин А. Ю. Комп’ютерне моделювання дискретно-подійних систем (ws04jg) : дистанційний навчальний курс. Платформа «Сікорський». URL: https://do.ipo.kpi.ua/course/view.php?id=2504 (дата звернення: 06.11.2026).

8. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python / P. Virtanen, R. Gommers, T. E. Oliphant et al. Nature Methods. 2020. Vol. 17. P. 261–272. DOI: https://doi.org/10.1038/s41592-019-0686-2.

9. ДСТУ 3008:2015. Інформація та документація. Звіти у сфері науки і техніки. Структура та правила оформлювання. [Чинний від 2017-07-01]. Вид. офіц. Київ : ДП «УкрНДНЦ», 2016. 26 с.

10. ДСТУ 8302:2015. Інформація та документація. Бібліографічне посилання. Загальні положення та правила складання. [Чинний від 2016-07-01]. Вид. офіц. Київ : ДП «УкрНДНЦ», 2016. 16 с.

11. BPMN 2.0 Symbols Reference. Camunda. URL: https://camunda.com/bpmn/reference/ (дата звернення: 06.11.2026).

12. Arena Simulation Software. Rockwell Automation. URL: https://www.rockwellautomation.com/en-us/products/software/arena-simulation.html (дата звернення: 06.11.2026).

13. Garnett O., Mandelbaum A., Reiman M. Designing a Call Center with Impatient Customers. Manufacturing & Service Operations Management. 2002. Vol. 4, no. 3. P. 208–227. DOI: https://doi.org/10.1287/msom.4.3.208.7753.

14. Welch P. D. The Statistical Analysis of Simulation Results. The Computer Performance Modeling Handbook / ed. S. S. Lavenberg. New York : Academic Press, 1983. P. 268–328.

15. Montgomery D. C. Design and Analysis of Experiments. 9th ed. Hoboken : John Wiley & Sons, 2017. 734 p.

16. Stetsenko I. V., Dyfuchyn A. Petri-object Simulation: Technique and Software. Information, Computing and Intelligent Systems. 2020. No. 1. P. 51–59. DOI: https://doi.org/10.20535/2708-4930.1.2020.216057.

ДОДАТОК А

Лістинг коду алгоритму імітації

У додатку наведено код алгоритму імітації мовою C# (.NET 10): елементи мережі МО, генератори випадкових величин, основний цикл, обчислення вихідних характеристик і протокол подій. Інтерфейс командного рядка (`Program.cs`) за вимогами до додатків не наведено; повний проєкт розміщено в репозиторії https://github.com/QuadRaliOn/kmdps-coursework-b12-airport-landing.

Лістинг А.1 – Вхідні змінні та параметри моделі (файл ModelParameters.cs)

```
namespace AirportSim;

public enum Dist
{
    Det,
    Exp,
    Unif,
    None
}

public sealed record ModelParameters
{
    public const double MinutesPerYear = 525_600.0;
    public const double RunwayCost = 3_000_000.0;
    public const int BaseRunways = 2;
    public const int MaxAdditionalRunways = 10;

    public double MeanInterarrival { get; init; } = 6.0;
    public Dist InterarrivalDist { get; init; } = Dist.Exp;

    public int Runways { get; init; } = BaseRunways;

    public double LandingOccupancy { get; init; } = 35.0;
    public Dist OccupancyDist { get; init; } = Dist.Det;

    public double RefuelThresholdMean { get; init; } = 70.0;
    public double RefuelThresholdDev { get; init; } = 10.0;
    public bool RefuelEnabled { get; init; } = true;

    public double RefuelCostMean { get; init; } = 1000.0;
    public double RefuelCostDev { get; init; } = 200.0;

    public double MaxWait { get; init; } = 140.0;
    public int MaxQueueLength { get; init; } = int.MaxValue;
    public Dist PatienceDist { get; init; } = Dist.Det;

    public double ProfitNoWait { get; init; } = 2000.0;
    public double ProfitWaitMean { get; init; } = 1500.0;
    public double ProfitWaitDev { get; init; } = 100.0;
```

```
    public int AdditionalRunways => Runways - BaseRunways;

    public override string ToString()
    {
        string refuel = RefuelEnabled ?
            $"unif({RefuelThresholdMean},{RefuelThresholdDev})" : "вимкнена";
        return $"m={Runways}, інтервал={InterarrivalDist}({MeanInterarrival}),
            " +
               $"смуга={OccupancyDist}({LandingOccupancy}), " +
               $"дозаправка={refuel}, відхід={PatienceDist}({MaxWait})";
    }
}
```

Лістинг А.2 – Генератори випадкових чисел і потоки RandomStreams (файл FunRand.cs)

```
namespace AirportSim;

public sealed class FunRand
{
    private readonly Random rnd;

    public FunRand(int seed) => rnd = new Random(seed);

    private double U01()
    {
        double u;
        do { u = rnd.NextDouble(); } while (u == 0.0);
        return u;
    }

    public double Exp(double mean) => -mean * Math.Log(U01());

    public double Unif(double mean, double dev) => mean - dev + 2.0 * dev *
        rnd.NextDouble();

    public double Sample(Dist dist, double mean, double dev = 0.0) => dist
        switch
    {
        Dist.Det => mean,
        Dist.Exp => Exp(mean),
        Dist.Unif => Unif(mean, dev),
        _ => double.PositiveInfinity
    };
}

public sealed class RandomStreams
{
    public RandomStreams(int seed)
    {
        Arrivals = new FunRand(unchecked(seed * 1_000_003 + 11));
        Attributes = new FunRand(unchecked(seed * 1_000_003 + 7_919));
        Service = new FunRand(unchecked(seed * 1_000_003 + 104_729));
    }

    public FunRand Arrivals { get; }
    public FunRand Attributes { get; }
```

```
    public FunRand Service { get; }
}
```

Лістинг А.3 – Запит мережі МО (файл Request.cs)

```
namespace AirportSim;

public class Request
{
    public Request(int id, double arrivalTime)
    {
        Id = id;
        ArrivalTime = arrivalTime;
    }

    public int Id { get; }
    public double ArrivalTime { get; }
    public double EnqueueTime { get; set; } = double.NaN;
    public double ServiceStart { get; set; } = double.NaN;
}
```

Лістинг А.4 – Запит задачі В12 – літак (файл Plane.cs)

```
namespace AirportSim;

public enum PlaneState
{
    Waiting,
    LandedNoWait,
    LandedAfterWait,
    Diverted
}

public sealed class Plane : Request
{
    public Plane(int id, double arrivalTime, double refuelThreshold, double
        refuelCost,
                 double patience, double profitAfterWait) : base(id,
                     arrivalTime)
    {
        RefuelThreshold = refuelThreshold;
        RefuelCost = refuelCost;
        Patience = patience;
        ProfitAfterWait = profitAfterWait;
        RefuelMoment = arrivalTime + refuelThreshold;
        LeaveMoment = arrivalTime + patience;
    }

    public double RefuelThreshold { get; }
    public double RefuelCost { get; }
    public double Patience { get; }
    public double ProfitAfterWait { get; }

    public double RefuelMoment { get; }
    public double LeaveMoment { get; }

    public bool Refueled { get; set; }
    public PlaneState State { get; set; } = PlaneState.Waiting;
}
```

Лістинг А.5 – Абстрактний елемент мережі МО (файл Element.cs)

```
namespace AirportSim;

public abstract class Element
{
    protected Element(string name) => Name = name;

    public string Name { get; }

    public double Tcurr { get; set; }

    public double Tnext { get; protected set; } = double.PositiveInfinity;

    public long Quantity { get; protected set; }

    public EventLog? Log { get; set; }

    public virtual void InAct(Request request) { }

    public abstract void OutAct();

    public virtual void DoStatistics(double delta) { }

    public virtual void ResetStatistics() => Quantity = 0;
}
```

Лістинг А.6 – Елемент надходження запитів (файл Create.cs)

```
namespace AirportSim;

public sealed class Create : Element
{
    private readonly ModelParameters p;
    private readonly FunRand arrivals;
    private readonly FunRand attributes;
    private int nextId = 1;

    public Create(ModelParameters p, RandomStreams streams) : base("C")
    {
        this.p = p;
        arrivals = streams.Arrivals;
        attributes = streams.Attributes;
        Tnext = arrivals.Sample(p.InterarrivalDist, p.MeanInterarrival);
    }

    public Element? Next { get; set; }

    public override void OutAct()
    {
        Quantity++;
        Plane plane = NewPlane();
        Tnext = Tcurr + arrivals.Sample(p.InterarrivalDist,
            p.MeanInterarrival);
        Next?.InAct(plane);
    }
```

```
    private Plane NewPlane()
    {
        double threshold = attributes.Unif(p.RefuelThresholdMean,
            p.RefuelThresholdDev);
        double refuelCost = attributes.Unif(p.RefuelCostMean,
            p.RefuelCostDev);
        double profit = attributes.Unif(p.ProfitWaitMean, p.ProfitWaitDev);
        double patience = p.PatienceDist switch
        {
            Dist.Det => p.MaxWait,
            Dist.Exp => attributes.Exp(p.MaxWait),
            _ => double.PositiveInfinity
        };
        return new Plane(nextId++, Tcurr, threshold, refuelCost, patience,
            profit);
    }
}
```

Лістинг А.7 – Універсальна черга FIFO (файл Queue.cs)

```
namespace AirportSim;

public sealed class QueueStatistics
{
    public long Enqueued;
    public long Exits;
    public long ServedExits;
    public double Area;
    public double WaitSumAll;
    public double WaitSumServed;
    public double MaxWaitServed;
    public int MaxLength;

    public void Reset(int currentLength)
    {
        Enqueued = Exits = ServedExits = 0;
        Area = WaitSumAll = WaitSumServed = MaxWaitServed = 0.0;
        MaxLength = currentLength;
    }
}

public class Queue : Element
{
    protected readonly LinkedList<Request> Items = new();

    public Queue(string name, int maxLength = int.MaxValue) : base(name) =>
        MaxLength = maxLength;

    public int MaxLength { get; }
    public int Length => Items.Count;
    public bool IsFull => Items.Count >= MaxLength;
    public QueueStatistics Stats { get; } = new();

    public override void InAct(Request request)
    {
        request.EnqueueTime = Tcurr;
        Items.AddLast(request);
        Stats.Enqueued++;
        if (Items.Count > Stats.MaxLength) Stats.MaxLength = Items.Count;
```

```
        UpdateTnext();
    }

    public Request? TakeFirst()
    {
        if (Items.First is not { } node) return null;
        Items.RemoveFirst();
        Request request = node.Value;
        double w = RecordExit(request);
        Stats.ServedExits++;
        Stats.WaitSumServed += w;
        if (w > Stats.MaxWaitServed) Stats.MaxWaitServed = w;
        UpdateTnext();
        return request;
    }

    protected double RecordExit(Request request)
    {
        double w = Tcurr - request.EnqueueTime;
        Quantity++;
        Stats.Exits++;
        Stats.WaitSumAll += w;
        return w;
    }

    public override void OutAct() { }

    protected virtual void UpdateTnext() { }

    public override void DoStatistics(double delta) => Stats.Area +=
        Items.Count * delta;

    public override void ResetStatistics()
    {
        base.ResetStatistics();
        Stats.Reset(Items.Count);
    }
}
```

Лістинг А.8 – Черга з обмеженим часом перебування – нащадок черги (файл TimedQueue.cs)

```
namespace AirportSim;

public sealed class TimedQueue : Queue
{
    private readonly bool refuelEnabled;
    private double nextRefuel = double.PositiveInfinity;
    private double nextLeave = double.PositiveInfinity;

    public TimedQueue(string name, bool refuelEnabled, int maxLength =
        int.MaxValue) : base(name, maxLength)
        => this.refuelEnabled = refuelEnabled;

    public Element? TimeoutExit { get; set; }

    public event Action<Plane>? Refueled;
    public event Action<Plane, double>? TimedOut;
```

```
public long Refuels { get; private set; }
public long TimeoutExits { get; private set; }
public long TimeoutExitsRefueled { get; private set; }
public double RefuelTimingError { get; private set; }

public override void OutAct()
{
    if (refuelEnabled && nextRefuel <= Tcurr)
    {
        foreach (Request request in Items)
            if (request is Plane { Refueled: false } plane &&
                plane.RefuelMoment <= Tcurr)
                Refuel(plane);
    }

    if (nextLeave <= Tcurr)
    {
        LinkedListNode<Request>? node = Items.First;
        while (node != null)
        {
            LinkedListNode<Request>? next = node.Next;
            if (node.Value is Plane plane && plane.LeaveMoment <= Tcurr)
            {
                Items.Remove(node);
                Leave(plane);
            }
            node = next;
        }
    }

    UpdateTnext();
}

private void Refuel(Plane plane)
{
    plane.Refueled = true;
    Refuels++;
    double err = Math.Abs(Tcurr - plane.RefuelMoment);
    if (err > RefuelTimingError) RefuelTimingError = err;
    Refueled?.Invoke(plane);
    Log?.Event(Tcurr,
        $"дозаправка, θ={plane.RefuelThreshold:F2},
        c={plane.RefuelCost:F2}", plane);
}

private void Leave(Plane plane)
{
    double w = RecordExit(plane);
    TimeoutExits++;
    if (plane.Refueled) TimeoutExitsRefueled++;
    TimedOut?.Invoke(plane, w);
    Log?.Event(Tcurr, $"відхід в інший аеропорт, w={w:F2}", plane);
    TimeoutExit?.InAct(plane);
}

protected override void UpdateTnext()
{
    nextRefuel = double.PositiveInfinity;
    nextLeave = double.PositiveInfinity;
    foreach (Request request in Items)
```

```
        {
            if (request is not Plane plane) continue;
            if (refuelEnabled && !plane.Refueled && plane.RefuelMoment <
                nextRefuel) nextRefuel = plane.RefuelMoment;
            if (plane.LeaveMoment < nextLeave) nextLeave = plane.LeaveMoment;
        }
        Tnext = Math.Min(nextRefuel, nextLeave);
    }

    public override void ResetStatistics()
    {
        base.ResetStatistics();
        Refuels = TimeoutExits = TimeoutExitsRefueled = 0;
        RefuelTimingError = 0.0;
    }
}
```

Лістинг А.9 – Універсальна багатоканальна СМО (файл Process.cs)

```
namespace AirportSim;

public sealed class Process : Element
{
    private readonly double[] channelTnext;
    private readonly Request?[] channelRequest;
    private readonly Func<double> serviceTime;

    public Process(string name, int channels, Queue queue, Func<double>
        serviceTime) : base(name)
    {
        channelTnext = Enumerable.Repeat(double.PositiveInfinity,
            channels).ToArray();
        channelRequest = new Request?[channels];
        Queue = queue;
        this.serviceTime = serviceTime;
    }

    public Queue Queue { get; }
    public Element? Next { get; set; }

    public event Action<Request, bool>? ChannelOccupied;

    public int Channels => channelTnext.Length;
    public int Busy { get; private set; }
    public long StartedDirect { get; private set; }
    public long StartedFromQueue { get; private set; }
    public long Failures { get; private set; }
    public double BusyArea { get; private set; }

    public override void InAct(Request request)
    {
        int r = Array.IndexOf(channelRequest, null);
        if (r >= 0)
        {
            Start(request, r, waited: false);
        }
        else if (Queue.IsFull)
        {
            Failures++;
            Log?.Event(Tcurr, "відмова: черга заповнена", request);
```

```
    }
    else
    {
        Queue.Tcurr = Tcurr;
        Queue.InAct(request);
        Log?.Event(Tcurr, "надходження → черга", request);
    }
    UpdateTnext();
}

public override void OutAct()
{
    for (int r = 0; r < channelTnext.Length; r++)
    {
        if (channelTnext[r] > Tcurr) continue;
        Complete(r);
        Queue.Tcurr = Tcurr;
        if (Queue.TakeFirst() is { } next)
            Start(next, r, waited: true);
    }
    UpdateTnext();
}

private void Start(Request request, int r, bool waited)
{
    request.ServiceStart = Tcurr;
    channelRequest[r] = request;
    channelTnext[r] = Tcurr + serviceTime();
    Busy++;
    if (waited) StartedFromQueue++;
    else StartedDirect++;
    ChannelOccupied?.Invoke(request, waited);
    Log?.Event(Tcurr, waited ?
        $"канал зайнято з черги, w={Tcurr - request.EnqueueTime:F2}"
                             : "надходження → канал", request, r);
}

private void Complete(int r)
{
    Request request = channelRequest[r]!;
    channelRequest[r] = null;
    channelTnext[r] = double.PositiveInfinity;
    Busy--;
    Quantity++;
    Log?.Event(Tcurr, "вихід з каналу", request, r);
    Next?.InAct(request);
}

private void UpdateTnext()
{
    double t = double.PositiveInfinity;
    foreach (double tc in channelTnext)
        if (tc < t) t = tc;
    Tnext = t;
}

public override void DoStatistics(double delta) => BusyArea += Busy *
    delta;

public override void ResetStatistics()
{
```

```
        base.ResetStatistics();
        StartedDirect = StartedFromQueue = Failures = 0;
        BusyArea = 0.0;
    }
}
```

Лістинг А.10 – Вихід із мережі (файл Dispose.cs)

```
namespace AirportSim;

public sealed class Dispose : Element
{
    public Dispose(string name) : base(name) { }

    public override void InAct(Request request) => Quantity++;

    public override void OutAct() { }
}
```

Лістинг А.11 – Облік доходу і витрат аеропорту (файл Accounting.cs)

```
namespace AirportSim;

public sealed class Accounting
{
    private readonly ModelParameters p;

    public Accounting(ModelParameters p, Process runways, TimedQueue queue)
    {
        this.p = p;
        runways.ChannelOccupied += OnLanding;
        queue.Refueled += OnRefuel;
        queue.TimedOut += (plane, _) => plane.State = PlaneState.Diverted;
    }

    public double Income { get; private set; }
    public double RefuelCost { get; private set; }

    private void OnLanding(Request request, bool waited)
    {
        var plane = (Plane)request;
        if (waited)
        {
            plane.State = PlaneState.LandedAfterWait;
            Income += plane.ProfitAfterWait;
        }
        else
        {
            plane.State = PlaneState.LandedNoWait;
            Income += p.ProfitNoWait;
        }
    }

    private void OnRefuel(Plane plane) => RefuelCost += plane.RefuelCost;

    public void Reset() => Income = RefuelCost = 0.0;
}
```

Лістинг А.12 – Мережа МО задачі В12 і алгоритм імітації (файл Model.cs)

```
namespace AirportSim;

public sealed class Model
{
    private readonly List<Element> elements;

    public Model(ModelParameters parameters, int seed, EventLog? log = null)
    {
        Parameters = parameters;
        Seed = seed;
        var streams = new RandomStreams(seed);
        Source = new Create(parameters, streams);
        Queue = new TimedQueue("Q", parameters.RefuelEnabled,
            parameters.MaxQueueLength);
        Runways = new Process("CMO", parameters.Runways, Queue,
                              () =>
streams.Service.Sample(parameters.OccupancyDist, parameters.LandingOccupancy));
        Landed = new Dispose("N1");
        OtherAirport = new Dispose("N2");
        Accounting = new Accounting(parameters, Runways, Queue);

        Source.Next = Runways;
        Runways.Next = Landed;
        Queue.TimeoutExit = OtherAirport;

        elements = new List<Element> { Runways, Queue, Source, Landed,
            OtherAirport };
        log?.Attach(this);
        foreach (Element e in elements) e.Log = log;
    }

    public ModelParameters Parameters { get; }
    public int Seed { get; }
    public Create Source { get; }
    public Process Runways { get; }
    public TimedQueue Queue { get; }
    public Dispose Landed { get; }
    public Dispose OtherAirport { get; }
    public Accounting Accounting { get; }

    public double Tcurr { get; private set; }

    public double StatStart { get; private set; }

    public long EventCount { get; private set; }

    public SimulationResult Simulate(double tmod, double twarm = 0.0,
        Action<Model>? afterEvent = null)
    {
        bool warmedUp = twarm <= 0.0;
        while (true)
        {
            double tnext = double.PositiveInfinity;
            foreach (Element e in elements)
                if (e.Tnext < tnext) tnext = e.Tnext;

            if (!warmedUp && tnext >= twarm)
```

```
            {
                Advance(Math.Min(twarm, tmod));
                foreach (Element e in elements) e.ResetStatistics();
                Accounting.Reset();
                StatStart = twarm;
                warmedUp = true;
            }

            if (tnext > tmod)
            {
                Advance(tmod);
                break;
            }

            Advance(tnext);

            foreach (Element e in elements)
                if (e.Tnext == Tcurr) e.OutAct();

            EventCount++;
            afterEvent?.Invoke(this);
        }
        return SimulationResult.From(this, tmod);
    }

    private void Advance(double t)
    {
        double delta = t - Tcurr;
        if (delta > 0)
            foreach (Element e in elements) e.DoStatistics(delta);
        Tcurr = t;
        foreach (Element e in elements) e.Tcurr = t;
    }
}
```

Лістинг А.13 – Вихідні характеристики прогону (файл SimulationResult.cs)

```
namespace AirportSim;

public sealed record SimulationResult
{
    public int Runways { get; init; }
    public int Seed { get; init; }
    public double Tmod { get; init; }
    public double Twarm { get; init; }
    public double Period { get; init; }

    public long Arrivals { get; init; }
    public long Enqueued { get; init; }
    public long LandedNoWait { get; init; }
    public long LandedAfterWait { get; init; }
    public long Refuels { get; init; }
    public long Diverted { get; init; }
    public long DivertedRefueled { get; init; }
    public long Released { get; init; }
    public double Income { get; init; }
    public double RefuelCost { get; init; }
    public double MeanQueue { get; init; }
    public int MaxQueue { get; init; }
    public double MeanBusy { get; init; }
```

```
public double MeanWaitLanded { get; init; }
public double MaxWaitLanded { get; init; }
public double QueueExitRate { get; init; }
public double MeanTimeInQueue { get; init; }
public int QueueAtEnd { get; init; }
public int BusyAtEnd { get; init; }
public double RefuelTimingError { get; init; }
public long Events { get; init; }
public long QueueExits { get; init; }
public long Failures { get; init; }
public double WaitSumAll { get; init; }

public long Landed => LandedNoWait + LandedAfterWait;
public double NetProfit => Income - RefuelCost;
public double AnnualProfit => NetProfit / Period *
    ModelParameters.MinutesPerYear;
public double LandingsPerYear => Landed / Period *
    ModelParameters.MinutesPerYear;
public double Utilization => MeanBusy / Runways;
public double ShareLandedAfterWait => Landed == 0 ? 0 :
    (double)LandedAfterWait / Landed;
public double ShareDiverted => Arrivals == 0 ? 0 : (double)Diverted /
    Arrivals;
public double ShareRefueled => Arrivals == 0 ? 0 : (double)Refuels /
    Arrivals;
public double MeanWaitAllLanded => Landed == 0 ? 0 : MeanWaitLanded *
    LandedAfterWait / Landed;
public double LittleQueue => QueueExitRate * MeanTimeInQueue;
public double ShareEnqueued => Arrivals == 0 ? 0 : (double)Enqueued /
    Arrivals;
public double MeanWaitPerArrival => Arrivals == 0 ? 0 : MeanTimeInQueue *
    QueueExitRate * Period / Arrivals;

public double MeanWaitAll => LandedNoWait + QueueExits == 0 ? 0 :
    WaitSumAll / (LandedNoWait + QueueExits);

public static SimulationResult From(Model model, double tmod)
{
    Process smo = model.Runways;
    TimedQueue q = model.Queue;
    QueueStatistics qs = q.Stats;
    double period = tmod - model.StatStart;
    return new SimulationResult
    {
        Runways = model.Parameters.Runways,
        Seed = model.Seed,
        Tmod = tmod,
        Twarm = model.StatStart,
        Period = period,
        Arrivals = model.Source.Quantity,
        Enqueued = qs.Enqueued,
        LandedNoWait = smo.StartedDirect,
        LandedAfterWait = smo.StartedFromQueue,
        Refuels = q.Refuels,
        Diverted = q.TimeoutExits,
        DivertedRefueled = q.TimeoutExitsRefueled,
        Released = smo.Quantity,
        Income = model.Accounting.Income,
        RefuelCost = model.Accounting.RefuelCost,
        MeanQueue = qs.Area / period,
        MaxQueue = qs.MaxLength,
        MeanBusy = smo.BusyArea / period,
        MeanWaitLanded = qs.ServedExits == 0 ? 0 : qs.WaitSumServed /
```

```
                qs.ServedExits,
            MaxWaitLanded = qs.MaxWaitServed,
            QueueExitRate = qs.Exits / period,
            MeanTimeInQueue = qs.Exits == 0 ? 0 : qs.WaitSumAll / qs.Exits,
            QueueAtEnd = q.Length,
            BusyAtEnd = smo.Busy,
            RefuelTimingError = q.RefuelTimingError,
            Events = model.EventCount,
            QueueExits = qs.Exits,
            Failures = smo.Failures,
            WaitSumAll = qs.WaitSumAll
        };
    }
}
```

Лістинг А.14 – Протокол подій (файл EventLog.cs)

```
using System.Globalization;

namespace AirportSim;

public sealed class EventLog
{
    private readonly TextWriter writer;
    private readonly int maxLines;
    private int lines;
    private Model? model;

    public EventLog(TextWriter writer, int maxLines = int.MaxValue)
    {
        this.writer = writer;
        this.maxLines = maxLines;
        writer.WriteLine("     t, хв | подія
            | літак |   a, хв | смуга |  L |   B  | N_r | N_d | прибуток,
            грн");
        writer.WriteLine(new string('-', 124));
    }

    public void Attach(Model m) => model = m;

    public void Event(double t, string description, Request request, int
        channel = -1)
    {
        if (lines++ >= maxLines || model is null) return;
        string ch = channel >= 0 ? $"D{channel + 1}" : "-";
        Process smo = model.Runways;
        TimedQueue q = model.Queue;
        double net = model.Accounting.Income - model.Accounting.RefuelCost;
        writer.WriteLine(string.Create(CultureInfo.InvariantCulture,
            $"{t,10:F2} | {description,-44} | {request.Id,5} |
                {request.ArrivalTime,7:F2} | {ch,5} | {q.Length,2} |
                {smo.Busy,2}/{smo.Channels,-2} | {q.Refuels,3} |
                {q.TimeoutExits,3} | {net,12:F2}"));
    }
}
```

ДОДАТОК Б

Лістинг коду проведення експериментів

Код верифікації та серій експериментів (C#) і статистична обробка результатів (Python 3, pandas, SciPy). Допоміжний код побудови рисунків (`analysis/plots.py`, matplotlib) не наведено; він є в репозиторії.

Лістинг Б.1 – Верифікація, перехідний період, серії експериментів (файл Experiments.cs)

```
using System.Globalization;
using System.Text;
using System.Text.Encodings.Web;
using System.Text.Json;
using System.Text.Json.Serialization;

namespace AirportSim;

public static class Experiments
{
    private static readonly CultureInfo Inv = CultureInfo.InvariantCulture;

    private static readonly JsonSerializerOptions JsonOptions = new()
    {
        WriteIndented = true,
        Encoder = JavaScriptEncoder.UnsafeRelaxedJsonEscaping,
        NumberHandling = JsonNumberHandling.AllowNamedFloatingPointLiterals,
        Converters = { new JsonStringEnumConverter() }
    };

    public static SimulationResult Run(ModelParameters p, int seed, double
        tmod, double twarm = 0.0, EventLog? log = null)
        => new Model(p, seed, log).Simulate(tmod, twarm);

    public static SimulationResult[] Replicate(ModelParameters p,
        IEnumerable<int> seeds, double tmod, double twarm)
        => seeds.AsParallel().AsOrdered().Select(s => Run(p, s, tmod,
            twarm)).ToArray();

    public static void Trace(string outDir)
    {
        Directory.CreateDirectory(outDir);

        using (var w = new StreamWriter(Path.Combine(outDir,
            "trace_random_m2.txt"), false, new UTF8Encoding(false)))
        {
            var p = new ModelParameters { Runways = 2 };
            w.WriteLine($"Протокол подій. {p}. Зерно 1, Tmod = 300 хв.");
            var r = Run(p, 1, 300.0, 0.0, new EventLog(w));
            w.WriteLine();
            w.WriteLine($"Підсумок: надійшло {r.Arrivals}, сіли без очікування
                {r.LandedNoWait}, після очікування {r.LandedAfterWait}, " +
                        $"дозаправок {r.Refuels}, відходів {r.Diverted}, у
                            черзі наприкінці {r.QueueAtEnd}.");
```

```
    }

    using (var w = new StreamWriter(Path.Combine(outDir,
        "trace_deterministic_m2.txt"), false, new UTF8Encoding(false)))
    {
        var p = DeterministicCheckParameters();
        w.WriteLine($"Протокол подій (детерміновані вхідні дані для ручної
            перевірки). {p}. Tmod = 420 хв.");
        var r = Run(p, 1, 420.0, 0.0, new EventLog(w));
        w.WriteLine();
        w.WriteLine($"Підсумок: надійшло {r.Arrivals}, сіли без очікування
            {r.LandedNoWait}, після очікування {r.LandedAfterWait}, " +
                    $"дозаправок {r.Refuels}, відходів {r.Diverted}, у
                        черзі наприкінці {r.QueueAtEnd}, " +
                    $"прибуток {r.NetProfit.ToString("F2", Inv)} грн.");
    }
}

public static ModelParameters DeterministicCheckParameters() => new()
{
    Runways = 2,
    InterarrivalDist = Dist.Det,
    RefuelThresholdDev = 0.0,
    RefuelCostDev = 0.0,
    ProfitWaitDev = 0.0
};

public static void Verify(string outDir)
{
    Directory.CreateDirectory(outDir);
    const double lambda = 1.0 / 6.0, mu = 1.0 / 35.0;
    var report = new Dictionary<string, object>();

    var v1 = new List<object>();
    foreach (int m in new[] { 6, 7, 8, 10 })
    {
        var p = new ModelParameters
        {
            Runways = m, OccupancyDist = Dist.Exp, PatienceDist =
                Dist.None, RefuelEnabled = false
        };
        var runs = Replicate(p, Enumerable.Range(1, 10), 2_050_000,
            50_000);
        var a = Analytic.ErlangC(m, lambda, mu);
        v1.Add(new
        {
            m,
            reps = runs.Length,
            tmod = 2_050_000,
            twarm = 50_000,
            sim = Summary(runs),
            analytic = a
        });
    }
    report["mmm_erlangC"] = v1;

    var v2 = new List<object>();
    foreach (int m in new[] { 2, 4, 6, 8 })
    {
        var p = new ModelParameters
        {
            Runways = m, OccupancyDist = Dist.Exp, PatienceDist =
```

```
            Dist.Exp, RefuelEnabled = false
    };
    var runs = Replicate(p, Enumerable.Range(1, 10), 1_010_000,
        10_000);
    var a = Analytic.ErlangA(m, lambda, mu, 1.0 / 140.0);
    v2.Add(new { m, reps = runs.Length, tmod = 1_010_000, twarm =
        10_000, sim = Summary(runs), analytic = a });
}
report["mmm_erlangA"] = v2;

var cases = new List<(string label, ModelParameters p)>
{
    ("базовий: m = 2", new ModelParameters()),
    ("m = 3", new ModelParameters { Runways = 3 }),
    ("m = 4", new ModelParameters { Runways = 4 }),
    ("m = 5", new ModelParameters { Runways = 5 }),
    ("m = 6", new ModelParameters { Runways = 6 }),
    ("m = 7", new ModelParameters { Runways = 7 }),
    ("m = 12", new ModelParameters { Runways = 12 }),
    ("m = 7, інтервал exp(5)", new ModelParameters { Runways = 7,
        MeanInterarrival = 5 }),
    ("m = 7, інтервал exp(7,5)", new ModelParameters { Runways = 7,
        MeanInterarrival = 7.5 }),
    ("m = 2, інтервал exp(12)", new ModelParameters { MeanInterarrival
        = 12 }),
    ("m = 2, інтервал exp(30)", new ModelParameters { MeanInterarrival
        = 30 }),
    ("m = 2, смуга 20 хв", new ModelParameters { LandingOccupancy = 20
        }),
    ("m = 2, w_max = 200 хв", new ModelParameters { MaxWait = 200 }),
    ("m = 2, інтервал 6 (детерм.)", new ModelParameters {
        InterarrivalDist = Dist.Det }),
    ("m = 6, θ = unif(100, 10)", new ModelParameters { Runways = 6,
        RefuelThresholdMean = 100 }),
    ("m = 6, без дозаправки", new ModelParameters { Runways = 6,
        RefuelEnabled = false }),
    ("m = 6, без відходу", new ModelParameters { Runways = 6,
        PatienceDist = Dist.None }),
    ("m = 6, c = unif(2000, 200)", new ModelParameters { Runways = 6,
        RefuelCostMean = 2000 }),
    ("m = 6, π₀  = 4000 грн", new ModelParameters { Runways = 6,
        ProfitNoWait = 4000 }),
    ("m = 6, π = unif(3000, 100)", new ModelParameters { Runways = 6,
        ProfitWaitMean = 3000 }),
};
var v3 = new List<object>();
foreach (var (label, p) in cases)
{
    var r = Run(p, 1, ModelParameters.MinutesPerYear, 0.0);
    long balance = r.Arrivals - r.Landed - r.Diverted - r.QueueAtEnd;
    v3.Add(new
    {
        label,
        parameters = p,
        result = r,
        conservation = balance,
        capacityPerYear = p.Runways * ModelParameters.MinutesPerYear /
            p.LandingOccupancy,
        demandPerYear = ModelParameters.MinutesPerYear /
            p.MeanInterarrival
    });
}
report["table"] = v3;
```

```
    var det = Run(DeterministicCheckParameters(), 1, 420.0);
    report["deterministic"] = det;

    File.WriteAllText(Path.Combine(outDir, "verification.json"),
        JsonSerializer.Serialize(report, JsonOptions));
    PrintVerification(v1, v2);
}

private static object Summary(SimulationResult[] runs) => new
{
    Lq = runs.Average(r => r.MeanQueue),
    LqSd = StdDev(runs.Select(r => r.MeanQueue)),
    Wq = runs.Average(r => r.MeanWaitPerArrival),
    PWait = runs.Average(r => r.ShareEnqueued),
    Utilization = runs.Average(r => r.Utilization),
    PAbandon = runs.Average(r => r.ShareDiverted),
    Throughput = runs.Average(r => r.Landed / r.Period),
    Little = runs.Average(r => r.LittleQueue)
};

private static void PrintVerification(List<object> v1, List<object> v2)
{
    Console.WriteLine("Верифікація збережена у verification.json");
    Console.WriteLine(JsonSerializer.Serialize(new { v1, v2 },
        JsonOptions));
}

public static void Transient(string outDir, string tag, int[]
    runwayCounts, int seeds, double tmax, double step)
{
    Directory.CreateDirectory(outDir);
    var rows = new System.Collections.Concurrent.ConcurrentBag<(int m, int
        seed, string line)>();
    var jobs = from m in runwayCounts from s in Enumerable.Range(1, seeds)
        select (m, s);
    jobs.AsParallel().ForAll(job =>
    {
        var model = new Model(new ModelParameters { Runways = job.m },
            job.s);
        var sb = new StringBuilder();
        double next = step;
        model.Simulate(tmax, 0.0, md =>
        {
            while (md.Tcurr >= next && next <= tmax)
            {
                sb.AppendLine(string.Join(",",
                    job.m, job.s, F(next), F(md.Tcurr),
                        F(md.Queue.Stats.Area), F(md.Runways.BusyArea),
                    F(md.Accounting.Income), F(md.Accounting.RefuelCost),
                        md.Source.Quantity,
                    md.Runways.StartedDirect +
                        md.Runways.StartedFromQueue,
                        md.Queue.TimeoutExits,
                    md.Queue.Refuels, md.Queue.Length));
                next += step;
            }
        });
        rows.Add((job.m, job.s, sb.ToString()));
    });
    using var w = new StreamWriter(Path.Combine(outDir,
        $"transient_{tag}.csv"), false, new UTF8Encoding(false));
```

```
w.WriteLine("m,seed,checkpoint,t,queueArea,busyArea,income,refuelCost,arrivals,l
anded,diverted,refuels,queueNow");
        foreach (var r in rows.OrderBy(r => r.m).ThenBy(r => r.seed))
            w.Write(r.line);
    }

    public static List<SimulationResult> Series(string outDir, string tag,
        ModelParameters baseParams, int reps,
                                                int seedBase, double twarm,
                                                    double tstat, int kmin =
                                                    0, int kmax = 10)
    {
        Directory.CreateDirectory(outDir);
        var jobs = (from k in Enumerable.Range(kmin, kmax - kmin + 1)
                    from r in Enumerable.Range(1, reps)
                    select (k, seed: seedBase + r)).ToList();
        var results = jobs.AsParallel().AsOrdered()
            .Select(j => Run(baseParams with { Runways =
                ModelParameters.BaseRunways + j.k }, j.seed, twarm + tstat,
                twarm))
            .ToList();

        string path = Path.Combine(outDir, $"series_{tag}.csv");
        using (var w = new StreamWriter(path, false, new UTF8Encoding(false)))
        {

w.WriteLine("tag,k,m,seed,interarrival,occupancy,twarm,period,arrivals,landedNoW
ait,landedAfterWait,refuels,diverted," +

"divertedRefueled,income,refuelCost,netProfit,annualProfit,meanQueue,maxQueue,me
anBusy,utilization," +

"meanWaitLanded,meanWaitAllLanded,maxWaitLanded,shareDiverted,shareRefueled,shar
eLandedAfterWait,events,meanWaitAll");
            foreach (var r in results)
                w.WriteLine(string.Join(",", tag, r.Runways -
                    ModelParameters.BaseRunways, r.Runways, r.Seed,
                    F(baseParams.MeanInterarrival),
                        F(baseParams.LandingOccupancy), F(r.Twarm),
                        F(r.Period), r.Arrivals,
                    r.LandedNoWait, r.LandedAfterWait, r.Refuels, r.Diverted,
                        r.DivertedRefueled, F(r.Income),
                    F(r.RefuelCost), F(r.NetProfit), F(r.AnnualProfit),
                        F(r.MeanQueue), r.MaxQueue, F(r.MeanBusy),
                    F(r.Utilization), F(r.MeanWaitLanded),
                        F(r.MeanWaitAllLanded), F(r.MaxWaitLanded),
                    F(r.ShareDiverted), F(r.ShareRefueled),
                        F(r.ShareLandedAfterWait), r.Events,
                        F(r.MeanWaitAll)));
        }
        PrintPayback(results, tag);
        return results;
    }

    public static void PrintPayback(List<SimulationResult> results, string
        tag)
    {
        var bySeed = results.GroupBy(r => r.Seed).ToDictionary(g => g.Key, g
            => g.ToDictionary(r => r.Runways));
        Console.WriteLine();
        Console.WriteLine($"Серія «{tag}»: {bySeed.Count} реплікацій на кожне
            k");
```

```
        Console.WriteLine("  k |  m |  p(m), млн грн/рік | Δp(k), млн грн/рік
            |  T(k), років |  T(k), діб");
        Console.WriteLine(new string('-', 84));
        foreach (int m in results.Select(r => r.Runways).Distinct().OrderBy(x
            => x))
        {
            int k = m - ModelParameters.BaseRunways;
            double pm = bySeed.Values.Average(d => d[m].AnnualProfit);
            string dp = "", T = "", Td = "";
            if (k > 0 && bySeed.Values.All(d =>
                d.ContainsKey(ModelParameters.BaseRunways)))
            {
                var t = bySeed.Values.Select(d => ModelParameters.RunwayCost *
                    k /
                                                    (d[m].AnnualProfit -

d[ModelParameters.BaseRunways].AnnualProfit)).ToList();
                double dpm = bySeed.Values.Average(d => d[m].AnnualProfit -
                    d[ModelParameters.BaseRunways].AnnualProfit);
                dp = (dpm / 1e6).ToString("F3", Inv);
                T = t.Average().ToString("F5", Inv);
                Td = (t.Average() * 365).ToString("F2", Inv);
            }
            Console.WriteLine($"{k,3} | {m,2} | {(pm / 1e6).ToString("F3",
                Inv),18} | {dp,18} | {T,12} | {Td,10}");
        }
    }

    private static string F(double x) => x.ToString("R", Inv);

    private static double StdDev(IEnumerable<double> xs)
    {
        var a = xs.ToArray();
        double mean = a.Average();
        return Math.Sqrt(a.Sum(x => (x - mean) * (x - mean)) / (a.Length -
            1));
    }
}
```

Лістинг Б.2 – Аналітичні формули M/M/m і M/M/m+M для верифікації (файл Analytic.cs)

```
namespace AirportSim;

public static class Analytic
{
    public sealed record QueueMetrics(double Lq, double Wq, double PWait,
        double Utilization,
                                      double PAbandon, double Throughput);

    public static QueueMetrics ErlangC(int m, double lambda, double mu)
    {
        double a = lambda / mu;
        double rho = a / m;
        if (rho >= 1.0) throw new
            ArgumentException("ρ ≥ 1: стаціонарного режиму немає");
        double term = 1.0, sum = 0.0;
        for (int n = 0; n < m; n++)
        {
            if (n > 0) term *= a / n;
```

```
            sum += term;
        }
        double termM = term * a / m;
        double tail = termM / (1.0 - rho);
        double p0 = 1.0 / (sum + tail);
        double pWait = tail * p0;
        double lq = pWait * rho / (1.0 - rho);
        return new QueueMetrics(lq, lq / lambda, pWait, rho, 0.0, lambda);
    }

    public static QueueMetrics ErlangA(int m, double lambda, double mu, double
        theta)
    {
        var w = new List<double> { 1.0 };
        double cur = 1.0;
        for (int n = 1; n < 1_000_000; n++)
        {
            double death = Math.Min(n, m) * mu + Math.Max(n - m, 0) * theta;
            cur *= lambda / death;
            w.Add(cur);
            if (n > m && cur < 1e-20) break;
        }
        double z = w.Sum();
        double lq = 0, busy = 0, pWait = 0;
        for (int n = 0; n < w.Count; n++)
        {
            double pn = w[n] / z;
            lq += Math.Max(n - m, 0) * pn;
            busy += Math.Min(n, m) * pn;
            if (n >= m) pWait += pn;
        }
        double pAbandon = theta * lq / lambda;
        return new QueueMetrics(lq, lq / lambda, pWait, busy / m, pAbandon,
            busy * mu);
    }
}
```

Лістинг Б.3 – Статистична обробка результатів експериментів (файл analyze.py)

```
import json
import sys
from pathlib import Path

import numpy as np
import pandas as pd
from scipy import stats

sys.path.insert(0, str(Path(__file__).resolve().parent))
import plots

sys.stdout.reconfigure(encoding="utf-8")
ROOT = Path(__file__).resolve().parents[1]
RES = ROOT / "results"

YEAR = 525_600.0
RUNWAY_COST = 3_000_000.0
BETA = 0.95
```

```
def ci_half(x, beta=BETA):
    x = np.asarray(x, float)
    n = len(x)
    return stats.t.ppf((1 + beta) / 2, n - 1) * x.std(ddof=1) / np.sqrt(n)

def payback_table(df):
    piv = df.pivot(index="seed", columns="k", values="annualProfit")
    rows = []
    for k in sorted(piv.columns):
        p = piv[k]
        row = {"k": int(k), "m": int(k) + 2, "n": int(p.count()),
               "p_mean": p.mean(), "p_sd": p.std(ddof=1), "p_ci": ci_half(p)}
        if k > 0:
            dp = piv[k] - piv[0]
            T = RUNWAY_COST * k / dp
            Ttot = RUNWAY_COST * k / piv[k]
            row.update({"dp_mean": dp.mean(), "dp_sd": dp.std(ddof=1),
                "dp_ci": ci_half(dp),
                        "T_mean": T.mean(), "T_sd": T.std(ddof=1), "T_ci":
                            ci_half(T),
                        "T_ratio": RUNWAY_COST * k / dp.mean(),
                        "Ttot": RUNWAY_COST * k / p.mean() if p.mean() > 0
                            else None})
        rows.append(row)
    return pd.DataFrame(rows), piv

def series_indicators(df):
    g = df.groupby("m")
    out = g.agg(meanQueue=("meanQueue", "mean"), maxQueue=("maxQueue", "max"),
        util=("utilization", "mean"),
                shareDiverted=("shareDiverted", "mean"),
                    shareRefueled=("shareRefueled", "mean"),
                shareWait=("shareLandedAfterWait", "mean"),
                    wait=("meanWaitLanded", "mean"),
                waitAll=("meanWaitAllLanded", "mean"), arrivals=("arrivals",
                    "mean"),
                landed0=("landedNoWait", "mean"), landedW=("landedAfterWait",
                    "mean"),
                refuels=("refuels", "mean"), diverted=("diverted", "mean"),
                income=("income", "mean"), refuelCost=("refuelCost", "mean"),
                events=("events", "mean"), waitAllPassed=("meanWaitAll",
                    "mean")).reset_index()
    return out

summary = {}

welch = pd.read_csv(RES / "transient_welch.csv", usecols=["m", "seed",
    "checkpoint", "queueNow"])
welch6 = pd.read_csv(RES / "transient_welch6.csv", usecols=["m", "seed",
    "checkpoint", "queueNow"])
WIN = 10



def welch_end(e, steady_from):
    v = e.values
    sm = np.empty_like(v, dtype=float)
    for i in range(len(v)):
        h = min(i, WIN, len(v) - 1 - i)
        sm[i] = v[i - h:i + h + 1].mean()
    ma = pd.Series(sm, index=e.index)
```

```
    tail = e[e.index >= steady_from]
    steady, noise = tail.mean(), tail.std()
    band = max(0.05 * steady, 2 * noise)
    ok = (ma - steady).abs() <= band
    t_w = next((int(t) for t in ma.index if ok[t]), None)
    return ma, steady, noise, band, t_w



warm = {}
smoothed = {}
curves = {int(m): (g, 10000, int(g.seed.nunique()), 20000) for m, g in
    welch.groupby("m") if m != 6}
curves[6] = (welch6, 20000, int(welch6.seed.nunique()),
    int(welch6.checkpoint.max()))
for m in sorted(curves):
    g, steady_from, reps, tmax = curves[m]
    e = g.groupby("checkpoint").queueNow.mean()
    ma, steady, noise, band, t_w = welch_end(e, steady_from)
    warm[m] = {"steady": steady, "noise": noise, "band": band, "t_warm": t_w,
        "reps": reps, "tmax": tmax,
               "steady_from": steady_from}
    smoothed[m] = ma
plots.plot_warmup(smoothed)
summary["warmup"] = {"replications": int(welch.seed.nunique()), "window": 2 *
    WIN + 1, "by_m": warm,
                     "chosen": 3000}


cum = pd.read_csv(RES / "transient_cum.csv")
cum["p"] = (cum.income - cum.refuelCost) / cum.t * YEAR / 1e6
cum["q"] = cum.queueArea / cum.t
conv = {}
for m, g in cum.groupby("m"):
    piv = g.pivot(index="checkpoint", columns="seed", values="p")
    rng = piv.max(axis=1) - piv.min(axis=1)
    conv[int(m)] = {}
    for eps in (5.0, 2.0, 1.0):
        ok = rng <= eps
        conv[int(m)][str(eps)] = next((int(t) for t in rng.index if
            ok[ok.index >= t].all()), None)
    conv[int(m)]["range_end"] = float(rng.iloc[-1])
    conv[int(m)]["p_end_mean"] = float(piv.iloc[-1].mean())
summary["runlength"] = conv


cum6 = pd.read_csv(RES / "transient_cum6.csv")
cum6["p"] = (cum6.income - cum6.refuelCost) / cum6.t * YEAR / 1e6
piv6 = cum6.pivot(index="checkpoint", columns="seed", values="p")
rng6 = piv6.max(axis=1) - piv6.min(axis=1)
long6 = {"tmax": int(piv6.index.max()), "seeds": int(piv6.shape[1])}
for eps in (5.0, 2.0, 1.0):
    ok = rng6 <= eps
    long6[str(eps)] = next((int(t) for t in rng6.index if ok[ok.index >=
        t].all()), None)
long6["range_by_year"] = {y: float(rng6[rng6.index <= y * YEAR].iloc[-1]) for
    y in range(1, 6)}
summary["runlength_m6_long"] = long6


plots.plot_cumulative(cum)


pilot = pd.read_csv(RES / "series_pilot.csv")
pt, _ = payback_table(pilot)
eps_T = 0.0005
```

```
eps_p = 1.0e6
need = []
for _, r in pt.iterrows():
    row = {"k": int(r.k), "m": int(r.m), "p_sd": r.p_sd, "n_p": 20 * (r.p_sd /
        eps_p) ** 2}
    if r.k > 0:
        row.update({"T_mean": r.T_mean, "T_sd": r.T_sd, "n_T": 20 * (r.T_sd /
            eps_T) ** 2})
    need.append(row)
summary["pilot"] = {"n0": int(pilot.seed.nunique()), "eps_T": eps_T, "eps_p":
    eps_p, "rows": need,
                    "n_T_max": max(x.get("n_T", 0) for x in need), "n_p_max":
                        max(x["n_p"] for x in need)}


main = pd.read_csv(RES / "series_main.csv")
mt, mpiv = payback_table(main)
ind = series_indicators(main)
summary["main"] = {"n": int(main.seed.nunique()), "table":
    mt.to_dict(orient="records"),
                   "indicators": ind.to_dict(orient="records")}
best = mt[mt.k > 0].sort_values("T_mean").iloc[0]
summary["main"]["best_k"] = int(best.k)
summary["main"]["best_T_years"] = best.T_mean
summary["main"]["best_T_days"] = best.T_mean * 365


Tmat = {int(k): RUNWAY_COST * k / (mpiv[k] - mpiv[0]) for k in mpiv.columns if
    k > 0}
kstar = int(best.k)
pairs = []
for k2 in sorted(Tmat):
    if k2 == kstar:
        continue
    d = Tmat[k2] - Tmat[kstar]
    t = d.mean() / (d.std(ddof=1) / np.sqrt(len(d)))
    pairs.append({"k": k2, "diff_mean": d.mean(), "diff_sd": d.std(ddof=1),
        "t": t,
                  "t_crit": stats.t.ppf(0.975, len(d) - 1), "p_value": 2 *
                      stats.t.sf(abs(t), len(d) - 1),
                  "share_reps_better": float((d > 0).mean())})
summary["main"]["paired"] = pairs


q = len(Tmat)
n = len(next(iter(Tmat.values())))
Y = np.array([Tmat[k].values for k in sorted(Tmat)])
ybar_j = Y.mean(axis=1)
ybar = Y.mean()
SS_f = n * ((ybar_j - ybar) ** 2).sum()
SS_r = ((Y - ybar_j[:, None]) ** 2).sum()
D_f = SS_f / (q - 1)
D_r = SS_r / (q * (n - 1))
F = D_f / D_r
F_crit = stats.f.ppf(0.95, q - 1, q * (n - 1))
s2 = Y.var(axis=1, ddof=1)
G = s2.max() / s2.sum()
f1, f2 = n - 1, q
G_crit = 1 / (1 + (f2 - 1) / stats.f.ppf(1 - 0.05 / f2, f1, (f2 - 1) * f1))
Yn = np.array([Tmat[k].values for k in (4, 5, 6)])
yj = Yn.mean(axis=1)
SSf_n = n * ((yj - Yn.mean()) ** 2).sum()
SSr_n = ((Yn - yj[:, None]) ** 2).sum()
F_n = (SSf_n / 2) / (SSr_n / (3 * (n - 1)))
summary["main"]["anova"] = {"q": q, "n": n, "SS_f": SS_f, "SS_r": SS_r, "D_f":
    D_f, "D_r": D_r, "F": F,
```

```
                                "F_crit": F_crit, "df1": q - 1, "df2": q * (n -
                                    1),
                                "cochran_G": G, "cochran_crit": G_crit,
                                "s2_max_k": int(sorted(Tmat)[int(s2.argmax())]),
                                "near": {"F": F_n, "F_crit": stats.f.ppf(0.95, 2,
                                    3 * (n - 1)),
                                         "D_f": SSf_n / 2, "D_r": SSr_n / (3 * (n
                                             - 1))}}


def cochran(arrs, alpha=0.05):
    s2_ = np.array([np.var(a, ddof=1) for a in arrs])
    q_, n_ = len(arrs), len(arrs[0])
    g = s2_.max() / s2_.sum()
    g_crit = 1 / (1 + (q_ - 1) / stats.f.ppf(1 - alpha / q_, n_ - 1, (q_ - 1)
        * (n_ - 1)))
    return float(g), float(g_crit)


def anova(arrs):
    Y_ = np.array(arrs)
    q_, n_ = Y_.shape
    yj_ = Y_.mean(axis=1)
    d_f = n_ * ((yj_ - Y_.mean()) ** 2).sum() / (q_ - 1)
    d_r = ((Y_ - yj_[:, None]) ** 2).sum() / (q_ * (n_ - 1))
    return {"q": q_, "df1": q_ - 1, "df2": q_ * (n_ - 1), "D_f": d_f, "D_r":
        d_r, "F": d_f / d_r,
            "F_crit": stats.f.ppf(0.95, q_ - 1, q_ * (n_ - 1))}


STABLE = list(range(5, 11))
lnT = [np.log(Tmat[k].values) for k in STABLE]
g_ln, gc_ln = cochran(lnT)
g_raw_st, gc_raw_st = cochran([Tmat[k].values for k in STABLE])
summary["main"]["cochran_steps"] = {
    "all_raw": {"G": G, "G_crit": G_crit},
    "stable_raw": {"G": g_raw_st, "G_crit": gc_raw_st},
    "stable_ln": {"G": g_ln, "G_crit": gc_ln},
    "cv_by_k": {int(k): float(Tmat[k].std(ddof=1) / Tmat[k].mean()) for k in
        sorted(Tmat)},
}
summary["main"]["anova_ln_stable"] = anova(lnT)


def anova_blocks(arrs):
    Y_ = np.array(arrs)
    q_, n_ = Y_.shape
    g = Y_.mean()
    ss_k = n_ * ((Y_.mean(axis=1) - g) ** 2).sum()
    ss_b = q_ * ((Y_.mean(axis=0) - g) ** 2).sum()
    ss_e = ((Y_ - g) ** 2).sum() - ss_k - ss_b
    df_k, df_b, df_e = q_ - 1, n_ - 1, (q_ - 1) * (n_ - 1)
    ms_k, ms_b, ms_e = ss_k / df_k, ss_b / df_b, ss_e / df_e
    return {"df_k": df_k, "df_b": df_b, "df_e": df_e, "MS_k": ms_k, "MS_b":
        ms_b, "MS_e": ms_e,
            "F_k": ms_k / ms_e, "F_k_crit": stats.f.ppf(0.95, df_k, df_e),
            "F_b": ms_b / ms_e, "F_b_crit": stats.f.ppf(0.95, df_b, df_e)}


summary["main"]["anova_blocks_ln_stable"] = anova_blocks(lnT)
```

```
def chi2_normal(x, alpha=0.05):
    x = np.asarray(x, float)
    n_ = len(x)
    kb = int(round(1 + 3.322 * np.log10(n_)))
    edges = np.linspace(x.min(), x.max(), kb + 1)
    obs, _ = np.histogram(x, edges)
    mu, sd = x.mean(), x.std(ddof=1)
    cdf = stats.norm.cdf(edges, mu, sd)
    cdf[0], cdf[-1] = 0.0, 1.0
    exp_ = n_ * np.diff(cdf)
    o, e_, co, ce = [], [], 0, 0.0
    for a, b in zip(obs, exp_):
        co += a
        ce += b
        if ce >= 5:
            o.append(co); e_.append(ce); co, ce = 0, 0.0
    if ce > 0:
        o[-1] += co; e_[-1] += ce
    o, e_ = np.array(o), np.array(e_)
    chi = float(((o - e_) ** 2 / e_).sum())
    dfree = len(o) - 1 - 2
    return {"n": n_, "mean": mu, "sd": sd, "min": float(x.min()), "max":
        float(x.max()), "cv": sd / mu,
            "bins": kb, "edges": edges.tolist(), "obs": obs.tolist(),
                "merged": int(len(o)), "chi2": chi,
            "df": dfree, "crit": float(stats.chi2.ppf(1 - alpha, dfree)),
            "crit01": float(stats.chi2.ppf(0.99, dfree)),
            "p_value": float(stats.chi2.sf(chi, dfree)), "shapiro_p":
                float(stats.shapiro(x).pvalue),
            "skew": float(stats.skew(x)), "kurt": float(stats.kurtosis(x))}



dist = {"T5_days": chi2_normal(Tmat[kstar].values * 365),
        "p7_mln": chi2_normal(mpiv[kstar].values / 1e6)}
summary["main"]["distribution"] = dist
plots.plot_hist(dist, kstar)
plots.plot_profit(mt)
plots.plot_payback(mt, kstar, best.T_mean)
plots.plot_service(ind)
plots.plot_boxplot(Tmat)


scen = {}
scen_tables = {}
for tag in ("lambda8", "main", "lambda12", "occ30"):
    df = pd.read_csv(RES / f"series_{tag}.csv")
    t, _ = payback_table(df)
    tt = t[t.k > 0]
    b = tt.sort_values("T_mean").iloc[0]
    scen[tag] = {"n": int(df.seed.nunique()), "best_k": int(b.k),
        "best_T_days": b.T_mean * 365,
                 "table": t.to_dict(orient="records"),
                 "indicators": series_indicators(df).to_dict(orient="records")}
    scen_tables[tag] = (tt, int(b.k))
plots.plot_sensitivity(scen_tables)
summary["scenarios"] = scen

scen_piv = {tag: pd.read_csv(RES / f"series_{tag}.csv").pivot(index="seed",
    columns="k",
```

```
values="annualProfit")
            for tag in ("lambda8", "main", "lambda12", "occ30")}



def two_level(a, b):
    a, b = np.asarray(a, float), np.asarray(b, float)
    g, gc = cochran([a, b])
    an = anova([a, b])
    d = b - a
    t = d.mean() / (d.std(ddof=1) / np.sqrt(len(d)))
    return {"mean_a": a.mean(), "mean_b": b.mean(), "sd_a": a.std(ddof=1),
        "sd_b": b.std(ddof=1),
            "cochran_G": g, "cochran_crit": gc, **{f"anova_{k}": v for k, v in
                an.items()},
            "paired_t": t, "t_crit": stats.t.ppf(0.975, len(d) - 1),
                "diff_mean": d.mean(),
            "diff_sd": d.std(ddof=1)}



def payback(piv, k):
    return RUNWAY_COST * k / (piv[k] - piv[0])



mod = {"profit_m7_vs_m6_occ30": two_level(scen_piv["main"][5] / 1e6,
    scen_piv["occ30"][4] / 1e6),
       "profit_anova_blocks": anova_blocks([scen_piv["main"][5].values / 1e6,
           scen_piv["occ30"][4].values / 1e6]),
       "T_best_vs_occ30_best_days": two_level(payback(scen_piv["main"], 5) *
           365,
                                              payback(scen_piv["occ30"], 4) *
                                                  365)}
sens = {}
for tag, piv in scen_piv.items():
    Ts = {k: payback(piv, k) * 365 for k in range(1, 11)}
    kb = min(Ts, key=lambda k: Ts[k].mean())
    row = {"best_k": kb, "T": {k: [float(Ts[k].mean()), float(ci_half(Ts[k]))]
        for k in range(3, 8)}}
    if kb != 5:
        d = Ts[5] - Ts[kb]
        row["k5_vs_best"] = {"diff": float(d.mean()),
                             "t": float(d.mean() / (d.std(ddof=1) /
                                 np.sqrt(len(d)))),
                             "share": float((d > 0).mean())}
    sens[tag] = row
grid_ln = [np.log(payback(scen_piv[tag], k).values)
           for tag in ("lambda8", "main", "lambda12") for k in range(3, 8)]
g_grid, gc_grid = cochran(grid_ln)
plan = {"cells": len(grid_ln), "n": len(grid_ln[0]), "cochran_G_lnT": g_grid,
    "cochran_crit": gc_grid}
summary["modification"] = mod
summary["sensitivity"] = sens
summary["plan_lambda_k"] = plan
print("Модифікація:", {k: {kk: round(float(vv), 5) for kk, vv in v.items()}
    for k, v in mod.items()})
print("Чутливість:", sens)



def pfe22(levels_lambda=(9.8, 10.2), levels_td=(34.3, 35.7), k=5, alpha=0.05):
    rows, cells, extra = [], [], {}
```

```
    for i, lam in enumerate(levels_lambda):
        for j, td in enumerate(levels_td):
            piv = pd.read_csv(RES /
                f"series_pfe_l{lam}_t{td}.csv").pivot(index="seed",
                columns="k",

values="annualProfit")
            cells.append(np.log(payback(piv, k).values))
            x1, x2 = 2 * i - 1, 2 * j - 1
            rows.append([1, x1, x2, x1 * x2])
            extra[f"{lam}/{td}"] = {kk: float(payback(piv, kk).mean() * 365)
                for kk in (3, 4, 5)}
    X = np.array(rows, float)
    N, n_ = len(cells), len(cells[0])
    ybar_ = np.array([c.mean() for c in cells])
    s2_rep = float(np.mean([c.var(ddof=1) for c in cells]))
    b = X.T @ ybar_ / N
    s_b = np.sqrt(s2_rep / (N * n_))
    g, gc = cochran(cells)
    return {"lambda": list(levels_lambda), "tD": list(levels_td), "k": k, "n":
        n_, "cochran_G": g,
            "cochran_crit": gc, "ybar": ybar_.tolist(), "T_days":
                (np.exp(ybar_) * 365).tolist(),
            "b": b.tolist(), "s_b": float(s_b), "t": (np.abs(b) /
                s_b).tolist(),
            "t_crit": float(stats.t.ppf(1 - alpha / 2, N * (n_ - 1))), "df": N
                * (n_ - 1),
            "s2_rep": s2_rep, "T_by_cell": extra}



def best_near_lambda(levels_lambda=(9.8, 10.2)):
    out = {}
    for lam in levels_lambda:
        piv = pd.read_csv(RES / f"series_pfe_l{lam}.csv").pivot(index="seed",
            columns="k", values="annualProfit")
        Ts = {kk: payback(piv, kk) * 365 for kk in range(1, 7)}
        kb = min(Ts, key=lambda kk: Ts[kk].mean())
        d = Ts[5] - Ts[kb]
        out[str(lam)] = {"best_k": kb, "T": {kk: float(Ts[kk].mean()) for kk
            in range(3, 7)},
                         "k5_minus_best": float(d.mean()),
                         "t": float(d.mean() / (d.std(ddof=1) /
                             np.sqrt(len(d)))) if kb != 5 else 0.0}
    return out



summary["pfe22"] = pfe22()
summary["best_near_lambda"] = best_near_lambda()
summary["main"]["T_total_profit"] = {int(kk): float((RUNWAY_COST * kk /
    mpiv[kk]).mean()) for kk in (2, 4, 5, 6)}
print("ПФЕ 2²:", summary["pfe22"])
print("λ ± 2 %:", summary["best_near_lambda"])
print("T за повним прибутком:", summary["main"]["T_total_profit"])

ver = json.loads((RES / "verification.json").read_text(encoding="utf-8"))
summary["verification"] = ver
plots.plot_verification(ver)



def clean(o):
    if isinstance(o, dict):
```

```
        return {str(k): clean(v) for k, v in o.items()}
    if isinstance(o, (list, tuple)):
        return [clean(v) for v in o]
    if isinstance(o, (np.floating,)):
        return float(o)
    if isinstance(o, (np.integer,)):
        return int(o)
    if isinstance(o, float) and (np.isnan(o) or np.isinf(o)):
        return None
    return o

(RES / "summary.json").write_text(json.dumps(clean(summary),
    ensure_ascii=False, indent=1), encoding="utf-8")

print("Перехідний період:", {m: v["t_warm"] for m, v in warm.items()})
print("Збіжність (ε = 1 млн):", {m: v["1.0"] for m, v in conv.items()})
print("Пілот: n_T_max = %.1f, n_p_max = %.1f" % (summary["pilot"]["n_T_max"],
    summary["pilot"]["n_p_max"]))
print(mt[["k", "m", "p_mean", "p_ci", "dp_mean", "T_mean", "T_ci", "T_ratio",
    "Ttot"]].to_string())
print("k* =", kstar, "T* =", best.T_mean, "років =", best.T_mean * 365, "діб")
for p_ in pairs:
    print("  k=%d: ΔT=%.6f, t=%.2f, p=%.3g, частка=%.2f"
          % (p_["k"], p_["diff_mean"], p_["t"], p_["p_value"],
             p_["share_reps_better"]))
print("ANOVA:", {k: v for k, v in summary["main"]["anova"].items() if k !=
    "near"},
      summary["main"]["anova"]["near"])
print("Cochran:", summary["main"]["cochran_steps"])
print("ANOVA ln:", summary["main"]["anova_ln_stable"])
print("Dist:", {k: {kk: v[kk] for kk in ("mean", "sd", "chi2", "df", "crit",
    "p_value", "shapiro_p", "obs")}
                for k, v in dist.items()})
print("Warm:", {m: v["t_warm"] for m, v in warm.items()})
print("Long m6:", long6)
for tag, s in scen.items():
    print(tag, "k* =", s["best_k"], "T =", round(s["best_T_days"], 2), "діб")
```